Journal of King Saud University - Computer and Information Sciences 36 (2024) 102254

HOSTED BY

Contents lists available at ScienceDirect

Journal of King Saud University - Computer and Information Sciences

journal homepage: www.sciencedirect.com

Full length article

# Deep reinforcement learning-based local path planning in dynamic environments for mobile robot☆

Bodong Tao, Jae-Hoon Kim *

*Department of Computer Engineering and Interdisciplinary Major of Maritime AI Convergence, National Korea Maritime and Ocean University, 727 Taejong-ro, Yeongdo-Gu, Busan 49112, South Korea*

ARTICLE INFO

*Keywords:*
Path planning
Deep reinforcement learning
Adaptive soft actor–critic
Mobile robots
Dynamic window approach
Tile coding

ABSTRACT

Path planning for robots in dynamic environments is a challenging task, as it requires balancing obstacle avoidance, trajectory smoothness, and path length during real-time planning.This paper proposes an algorithm called Adaptive Soft Actor–Critic (ASAC), which combines the Soft Actor–Critic (SAC) algorithm, tile coding, and the Dynamic Window Approach (DWA) to enhance path planning capabilities. ASAC leverages SAC with an automatic entropy adjustment mechanism to balance exploration and exploitation, integrates tile coding for improved feature representation, and utilizes DWA to define the action space through parameters such as target heading, obstacle distance, and velocity In this framework, the action space is defined by DWA's three weighting parameters: target heading deviation, distance to the nearest obstacle, and velocity. To facilitate the learning process, a non-sparse reward function is designed, incorporating factors such as Time-to-Collision (TTC), heading, and velocity. To validate the effectiveness of the algorithm, experiments were conducted in four different environments, and the algorithm was evaluated based on metrics such as trajectory deviation, smoothness, and time to reach the end point. The results demonstrate that ASAC outperforms existing algorithms in terms of trajectory smoothness, arrival time, and overall adaptability across various scenarios, effectively enabling path planning in dynamic environments.

## 1. Introduction

With the rapid advancements in fields such as computer science, the Internet of Things (IoT), and Artificial Intelligence (AI), significant progress has been made in the research of mobile robots, which are increasingly being applied in areas such as autonomous vehicles, Autonomous Underwater Vehicles (AUVs), and Unmanned Aerial Vehicles (UAVs). Path planning is a crucial aspect of mobile robots, enabling safe and effective navigation to designated end points, which is vital for diverse and complex environments.

Path planning in robotic navigation is broadly divided into global path planning and local path planning, each with specific application scenarios and technical challenges (Sanchez-Ibanez et al., 2021). Global path planning utilizes comprehensive environmental information, including start and end points as well as obstacle positions, to identify an optimal and obstacle-free path. However, global path planning typically requires recalculating when environmental changes occur. This recalculation process limits the ability to respond promptly to environmental changes, which generally results in a lack of real-time capability in global path planning. In contrast, local path planning uses the robot's real-time sensor data to navigate unknown or partially known environments, allowing it to dynamically detect and adapt to obstacles as environmental conditions change. This approach requires immediate responsiveness, particularly when dealing with dynamic obstacles. Local path planning algorithms must continuously adjust to changes in obstacle positions, whether static or dynamic, to ensure safe and efficient navigation.

While global path planning provides an overall path through the environment and local path planning enables real-time adjustments

☆ This work was supported by the BK21 Four program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education of Korea (Center for Creative Leaders in Maritime Convergence).

* Corresponding author.
*E-mail address:* jhoon@kmou.ac.kr (J.-H. Kim).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2024.102254
Received 9 September 2024; Received in revised form 11 November 2024; Accepted 14 November 2024
Available online 22 November 2024

to nearby obstacles, conventional algorithms still face challenges in adapting to dynamic environments. Reinforcement Learning (RL) offers a promising solution, as it enables an agent to adapt its behavior based on feedback, learning optimal policies through real-time interactions with the environment (Naeem et al., 2020). This adaptability makes RL a suitable framework for improving robot navigation in changing conditions. However, conventional algorithms, including RL-based algorithms, often encounter limitations in dynamic environments due to challenges in handling high-dimensional state spaces and sparse reward structures.

To address these limitations, Deep Reinforcement Learning (DRL) extends the capabilities of RL by combining it with deep neural networks, which can process high-dimensional state spaces and complex tasks. DRL algorithms use neural networks to approximate value and policy functions, allowing the agent to generalize better across diverse states and adapt to complex, variable environments. In this context, the Soft Actor-Critic (SAC) algorithm has shown particular promise, as it incorporates a policy entropy term to encourage exploration, balancing exploration, and exploitation to improve stability and learning efficiency (Yang et al., 2023). In subsequent developments of the SAC algorithm, an automatic entropy adjustment mechanism was introduced to dynamically tune entropy based on environmental conditions, enhancing SAC's adaptability and making it more suitable for mobile robot path planning in dynamic environments (Chen et al., 2023).

However, path planning in real-world dynamic environments requires not only adaptive learning but also efficient path generation to handle immediate obstacles and constraints. To address this need, the Dynamic Window Approach (DWA) is widely used as a core method for local path planning. DWA operates by sampling feasible velocity combinations and predicting the resulting positions over a short time horizon, allowing for the generation of smooth and collision-free paths (Reda et al., 2024).

Integrating DRL with DWA leverages DRL's learning ability to enhance DWA's adaptability and responsiveness, making local path planning more robust in dynamic environments (Lou et al., 2024). Additionally, sparse reward structures in complex environments can hinder learning efficiency, as limited feedback slows the agent's progress toward optimal solutions (Zeng et al., 2019). Another challenge lies in limited feature representation, which may prevent the model from accurately capturing critical state information needed for effective decision-making. Methods like tile coding have been applied to enhance feature representation, allowing the agent to distinguish relevant states more accurately and thereby improve overall path planning performance (Abdoos et al., 2014).

To address these interconnected challenges, this paper proposes a novel path planning method that combines the SAC algorithm, DWA and tile coding for mobile robot path planning in dynamic environments. SAC is a DRL algorithm based on entropy-regularized policy gradients and integrates an automatic entropy adjustment mechanism to balance exploration and exploitation. This adjustment mechanism helps address the challenge of adapting robot behavior to changing environments. Meanwhile, DWA, effective for collision avoidance by evaluating velocity, distance, and heading, optimizes real-time velocity selection under dynamic conditions. Additionally, tile coding is applied for feature representation to improve the accuracy of critical state estimation, while a non-sparse reward function addresses the issues of sparse rewards in traditional path planning, further enhancing the effectiveness and safety of the planning process. To validate the effectiveness of the proposed method, we conducted simulation experiments across various dynamic scenarios. We compared the performance of our approach with existing baseline methods, including DDPG, PPO, SAC, and SAC*. The results of the experiments indicate that our method demonstrates advantages over these baseline methods in planning efficiency and safety, illustrating its effectiveness in path planning in dynamic environments.

The specific contributions are as follows: (1) Combining the SAC algorithm with an automatic entropy adjustment mechanism and DWA to enhance path planning capabilities, making robot path planning in dynamic environments more efficient. (2) Integrating tile coding for feature representation to improve the representation of critical states, enhancing state estimation accuracy and further improving path planning performance. (3) Designing a non-sparse reward function that includes Time-to-Collision (TTC), while evaluating velocity and heading, addressing the issue of sparse rewards in path evaluation and facilitating safer and more efficient path planning. (4) Proposing a novel path evaluation metric called average path deviation to provide a more precise assessment of path efficiency by measuring the robot's deviation from the optimal path.

The structure of the remaining sections of this paper is as follows: Section 2 reviews related work in conventional, heuristic, and RL-based path planning methods. Section 3 presents an overview of the fundamental principles of DWA and reinforcement learning. Section 4 explores the integration of the SAC algorithm with the automatic entropy adjustment mechanism and the DWA algorithm, along with the application of tile coding and a non-sparse reward function in path planning. Section 5 discusses the experimental setup, presents the results, and provides analysis and discussion. Finally, Section 6 summarizes the key findings and discusses directions for future research.

## 2. Related work

Path planning and obstacle avoidance remain pivotal challenges in the field of mobile robots. To address these challenges, researchers have developed numerous algorithms that enable robots to navigate safely and efficiently under various environmental conditions. Conventional algorithms include graph search-based algorithms, such as the A* algorithm (Guruji et al., 2016) and Dijkstra's algorithm (Luo et al., 2020), which determine the optimal path by searching a graph structure. However, these algorithms depend on complete map information and may lose efficiency in complex or dynamic environments. Additionally, sampling-based algorithms such as Rapidly-Exploring Random Trees (RRT) (Melchior and Simmons, 2007) and Probabilistic Roadmaps (PRM) (Li et al., 2022) generate nodes randomly to construct paths, which can be beneficial for handling high-dimensional spaces and complex environments. However, they usually cannot guarantee the optimal path, and their efficiency and path quality are influenced by the sampling strategy and node density. Furthermore, with the development of swarm intelligence algorithms, heuristic algorithms like Ant Colony Optimization (ACO) (Li et al., 2019), Genetic Algorithms (GA) (Tam et al., 2019), and Particle Swarm Optimization (PSO) (Tao and Kim, 2024) have been widely applied to global path planning. These algorithms are relatively straightforward to implement and can generate paths quickly, but they may face challenges such as converging to local optima and limited adaptability to real-time changes.

However, conventional path planning and swarm intelligence algorithms tend to neglect kinematic constraints, which are essential for accurate path following. Even if a path is planned, the robot may not accurately follow the route without considering kinematic constraints. When algorithms incorporate these constraints, mobile robots are more likely to follow the predetermined route accurately, as these constraints ensure the path is feasible for the robot's dynamics.

The consideration of kinematic constraints is increasingly important. In scenarios requiring real-time or near-real-time path planning, Model Predictive Control (MPC) (Liu et al., 2017) and the DWA (Henkel et al., 2016) are well-suited. These algorithms can adjust the robot's trajectory in response to environmental changes and constraints. However, they are sensitive to parameter settings, and improper configurations may result in suboptimal paths, such as overly conservative or aggressive obstacle avoidance, thereby complicating dynamic obstacle avoidance.

In the context of conventional and heuristic algorithms facing limitations in dynamic environments, RL presents a viable solution. RL as a machine learning method, relies on interactions between an agent and its environment to learn optimal policies. Through these interactions and by observing the consequences of actions, the agent adjusts its behavior based on feedback to maximize cumulative rewards (Arulkumaran et al., 2017). In mobile robot path planning, RL can adapt to environmental changes and optimize long-term policies. In RL, the agent's policy is refined through a reward function, which provides feedback after each action taken. This feedback can be positive, negative, or neutral. However, designing the reward function poses a classic challenge: sparse rewards (Schoettler et al., 2020). In cases of sparse rewards, the agent receives infrequent feedback, with many actions yielding a reward of 0. This lack of useful feedback over extended periods makes it difficult for the agent to improve its performance and eventually converge on an optimal solution (Ladosz et al., 2022). Maoudj and Hentout (2020) applied Q-learning to path planning by equipping the robot with prior knowledge through a designed reward function, aiding in quick convergence and enhancing optimization. Xie et al. (2019) proposed two new non-sparse reward functions to improve the efficiency of robot path planning in unstructured environments with obstacles. Therefore, a well-designed reward function is crucial for guiding the agent to develop an effective policy to achieve its goals.

However, conventional RL encounters challenges when dealing with high-dimensional state spaces and complex tasks, including low sample efficiency and difficulties in handling continuous action spaces (Zhong et al., 2019). To address these issues, DRL was introduced (Wang et al., 2022). DRL combines deep learning techniques with RL methods to handle high-dimensional state spaces and complex tasks. In DRL, deep neural networks approximate the value functions and policy functions in RL. The value function assesses the expected return of an action or policy in a given state, while the policy function determines the action to take in a specific state. This integration allows DRL algorithms to handle high-dimensional state spaces effectively and improve decision-making by automatically extracting important features. Thus, DRL enhances learning efficiency and shows increased potential for improving performance in complex environments, making it widely applicable to path planning. Sasaki et al. (2019) proposed a path planning method for mobile robot path planning in dynamic environments based on the Asynchronous Advantage Actor–Critic (A3C) algorithm. They introduced a composite learning method combining the A3C algorithm, Long Short-Term Memory (LSTM) network, and Q-learning to improve collision avoidance efficiency in unknown environments.

Recently, an efficient DRL method, the SAC algorithm, has gained attention for incorporating an entropy term into policy optimization to encourage exploration (Xu et al., 2023). SAC improves the stability and efficiency of the learning process by balancing exploration and exploitation, making it well-suited for handling complex tasks like mobile robot path planning (Yang et al., 2022). Moreover, integrating DRL with conventional methods can enhance algorithm efficiency, improve sample quality, and accelerate training convergence. For instance, Li et al. (2021) integrated the Artificial Potential Field (APF) algorithm with the Deep Q-Network (DQN) to refine the action space and reward function, addressing the path planning problem for Unmanned Surface Vehicles (USVs). Similarly, Wu et al. (2023) proposed a method combining Proximal Policy Optimization (PPO) and DWA for obstacle avoidance in Maritime Autonomous Surface Ships (MASS), significantly improving obstacle avoidance efficiency in complex maritime environments. While DRL's strong feature extraction capability enhances RL's generalization and robustness, the sensitivity of neural networks to hyperparameters can still lead to potential overfitting and interference during training (Liu et al., 2019).

In response to these challenges encountered in DRL, tile coding has proven to be a beneficial approach. Ghiassian et al. (2020) improved input preprocessing using tile coding, mapping the observed state space to a higher-dimensional space to reduce inappropriate generalization, significantly improving learning speed in several standard neural network systems. Additionally, Abdulhai et al. (2003) demonstrated the application of tile coding in complex traffic signal control tasks, proving its adaptability and versatility in high-dimensional state spaces.

Despite advancements in conventional, heuristic, and RL-based path planning methods, challenges persist in achieving robust and efficient navigation in dynamic environments. Limitations such as sensitivity to parameter settings, sparse reward structures, and the inability to consistently address kinematic constraints underscore the need for improved methods. To address these limitations, this paper proposes the ASAC algorithm, which enhances adaptability and efficiency, providing a robust solution for complex, dynamic environments.

## 3. Preliminaries

In dynamic environments, path planning for mobile robots involves sequential decision-making to navigate from the start to the end point. This process is typically classified as local path planning, which relies on real-time sensor data to dynamically identify and avoid obstacles, ensuring the robot's safe and efficient movement in constantly changing environments. The presence of both static and dynamic obstacles increases the complexity and uncertainty of path planning, posing significant challenges for safe operation. Given the challenges posed by dynamic environments, particularly the need for real-time decision-making and adaptability, this section introduces DWA and RL as fundamental methods that address these challenges.

DWA is a widely used local path planning algorithm in robotics. It calculates a set of possible velocity combinations (linear and angular velocities) within the robot's current velocity range and selects the optimal command. By integrating the robot's dynamic constraints, the DWA algorithm efficiently generates smooth paths for obstacle avoidance. However, its reliance on pre-set parameters may limit its adaptability in complex environments due to the algorithm's sensitivity to these settings. To address this limitation, RL can be employed to automatically learn and optimize DWA's parameters, allowing it to adapt to varying environments with suitable parameter configurations. By integrating DWA with RL, robots can learn optimal path planning strategies in complex and dynamic environments, consequently enhancing safety and efficiency in path planning. We will further elaborate on the concepts of DWA and RL in the subsequent sections.

### 3.1. Dynamic window approach

Operating under predictive control, the DWA algorithm systematically executes velocity sampling, trajectory prediction, and evaluation. It begins by sampling velocity combinations $(v, w)$, where $v$ represents the linear velocity and $w$ represents the angular velocity of the robot. The pair $(v, w)$ indicates the set of velocity combinations that the robot can choose from over a future period. These velocities are confined within the linear velocity range $[v_{min}, v_{max}]$ and the angular velocity range $[w_{min}, w_{max}]$, where $v_{min}$, $v_{max}$, $w_{min}$, and $w_{max}$ are the minimum linear velocity, maximum linear velocity, minimum angular velocity, and maximum angular velocity, respectively, as determined by the robot's dynamic constraints. This sampling considers the velocity and position of the robot, along with a defined prediction time interval $\Delta t$, while adhering to hardware constraints like maximum acceleration (Wang et al., 2023). These velocity samples constitute the algorithm's "dynamic window", adapting to dynamic and hardware constraints. After establishing the dynamic window, the algorithm predicts possible movement trajectories based on the selected velocity combinations. The trajectories are then evaluated using an objective function, and the velocity combination corresponding to the optimal trajectory is chosen as the movement command for the mobile robot. This process repeats until the robot reaches the end point. In the DWA algorithm, the first step is velocity sampling. During path planning, the DWA algorithm

considers the robot's hardware and environmental constraints. The primary consideration for velocity sampling is the robot's velocity limits, aiming to generate a series of possible velocity combinations within the preset dynamic window. The velocity limits are categorized as follows:

$$v_m = \left\{(v, w) | v \in [v_{min}, v_{max}], w \in [w_{min}, w_{max}]\right\}, \tag{1}$$

where this range defines all feasible combinations of linear and angular velocities that the robot can achieve.

Additionally, due to the limitations of the drive motors, there are boundary constraints on the acceleration of the robot's linear and angular velocities. Therefore, when considering acceleration, the velocity space that can be sampled, $v_d$, is defined as:

$$v_d = \left\{(v, w) \middle| v \in [v_c - a_{v_{\max}} \cdot \Delta t, v_c + a_{v_{\max}} \cdot \Delta t], \right. \\ \left. w \in [w_c - a_{w_{\max}} \cdot \Delta t, w_c + a_{w_{\max}} \cdot \Delta t] \right\}, \tag{2}$$

where $v_c$ and $w_c$ correspond to the current linear and angular velocities of the mobile robot, respectively. Similarly, $a_{v_{\max}}$ and $a_{w_{\max}}$ correspond to the robot's maximum linear and angular accelerations, respectively.

In local planning, real-time obstacle avoidance is achieved by considering the dynamics of surrounding obstacles. Therefore, at any given moment, the constraint condition for the robot to avoid collisions with surrounding obstacles is:

$$v_a = \left\{(v, w) \middle| v \in \left[v_{\min}, \sqrt{2 \cdot \text{dist}(v, w) \cdot a_{v_{\max}}}\right], \right. \\ \left. w \in \left[w_{\min}, \sqrt{2 \cdot \text{dist}(v, w) \cdot a_{w_{\max}}}\right] \right\}, \tag{3}$$

where $dist(v, w)$ represents the distance between the robot's predicted position, considering all possible velocity combinations $v$ and $w$, and the nearest obstacle over a future period. This period is typically set as an integer multiple of the time step. In scenarios without obstacles, $dist(v, w)$ would be a constant.

The purpose of this formula is to evaluate the safety of the robot under different velocity and angular velocity combinations. By calculating $dist(v, w)$, it is possible to determine the minimum distance between the robot and obstacles within a given time step, thereby selecting a velocity and angular velocity combination that allows the robot to avoid obstacles effectively while moving towards the end point. Finally, velocity sampling needs to be performed at the intersection of the three velocity spaces, $v_m$, $v_d$, and $v_a$. This ensures that the selected velocity combinations satisfy the robot's motion constraints and can effectively avoid obstacles while moving toward the end point.

Once the sampling space is determined, the DWA algorithm uniformly samples the space at a specified sampling interval. In the velocity space, the number of samples for linear and angular velocity are set. For example, if $n_v$ points are sampled within the linear velocity range and $n_w$ points within the angular velocity range, a total of $n = n_v \cdot n_w$ velocity combinations are obtained. For each velocity combination $(v, w)$, the robot's current position and orientation are used as the basis to simulate the movement trajectory for each combination over the prediction time, generating and recording a series of trajectory points. These trajectories reflect the robot's movement paths under different velocity combinations and prepare for the next trajectory evaluation step to select the optimal velocity combination $(v_{best}, w_{best})$.

The final step is trajectory evaluation. The goal of trajectory evaluation is to select the optimal velocity combination, ensuring the robot can move safely and efficiently toward the end point. Each predicted trajectory is evaluated holistically, considering factors such as velocity, distance to the nearest obstacle, and deviation from the target heading.

Thus, the ideal trajectory seeks to balance the following conditions: higher velocity is preferred, greater distance from obstacles is beneficial, and smaller angle deviation from the target direction is desirable. The following objective function is used as the trajectory evaluation formula:

$$G(v, w) = \sigma(w_{head} \cdot head(v, w) + w_{dist} \cdot dist(v, w) + w_{vel} \cdot vel(v, w)), \tag{4}$$

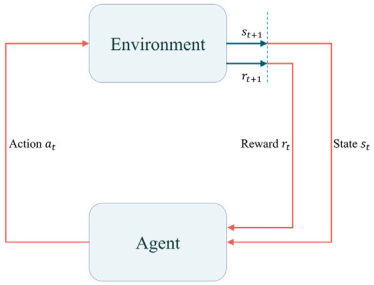


**Fig. 1.** Interaction loop between the agent and the environment in reinforcement learning.

where $w_{head}$, $w_{dist}$, and $w_{vel}$ are the weights for the target heading deviation, distance to the nearest obstacle, and velocity, respectively. Here, $head(v, w)$ represents the deviation of the robot's heading from the target direction, while $vel(v, w)$ represents the linear velocity for the given velocity combination $(v, w)$.

Since the process of local path planning involves multi-sensor data collection, and the information collected may be discontinuous, normalization is applied to minimize evaluation differences. Here, $\sigma$ represents normalization. The normalization process is shown as follows:

$$\sigma(\text{head}(v_i, w_i)) = \frac{\text{head}(v_i, w_i)}{\sum_{j=1}^{n} \text{head}(v_j, w_j)}, \tag{5}$$

$$\sigma(\text{dist}_i(v, w_i)) = \frac{\text{dist}(v_i, w_i)}{\sum_{j=1}^{n} \text{dist}(v_j, w_j)}, \tag{6}$$

$$\sigma(\text{vel}_i(v_i, w_i)) = \frac{\text{vel}(v_i, w_i)}{\sum_{j=1}^{n} \text{vel}(v_j, w_j)}, \tag{7}$$

where $v_i$ and $w_i$ represent the linear and angular velocities for the $i$th simulated trajectory, respectively. Here, $i$ represents the $i$th trajectory among the set of all possible velocity combinations, and $n$ is the total number of these velocity combinations that satisfy the robot's dynamic constraints.

DWA generates a series of possible velocity combinations based on the robot's current velocity and kinematic constraints, evaluates the trajectories of these combinations over a future period, and then selects the optimal velocity combination to achieve path planning. While DWA performs real-time path planning efficiently but may be limited in dynamic environments by preset parameters, such as the weights for target heading deviation, distance to the nearest obstacle, and velocity. These weights directly influence the quality of the paths generated by the DWA algorithm.

### 3.2. Reinforcement learning

RL is a systematic approach to goal-directed learning and decision-making, facilitated by interactive feedback mechanisms (Balleine et al., 2015). Specifically, RL involves an agent continuously interacting with the environment, receiving feedback in the form of a reward function, and adjusting its policy to maximize cumulative rewards (Busoniu et al., 2008). This interaction loop between the agent and the environment is illustrated in Fig. 1.

In RL, the agent assesses the current state at each timestep, making decisions by selecting appropriate actions based on this state (Whitehead and Lin, 1995). The state $s$ represents the agent's specific situation at a given time, the action $a$ is the operation that the agent can perform in that state, and the reward $r$ is the feedback signal given by the environment after the agent performs the action.

The agent learns the optimal policy through trial and error by selecting actions based on the current state. A policy refers to the probability distribution of action choices given a state. RL does not require labeled data; instead, it learns the best behavior autonomously through the

set reward function (Kiran et al., 2021). Therefore, a unique challenge in RL is balancing exploration and exploitation. Exploration involves trying new actions to discover better strategies, while exploitation involves choosing the current best action based on past experience.

A Markov Decision Process (MDP) is the mathematical framework that describes RL problems. An MDP systematically defines the agent's behavior and objectives in the environment, providing a theoretical foundation for designing and optimizing RL algorithms (Zhang et al., 2021). MDP includes the following elements: state space, action space, state transition probability, reward function, and discount factor, usually represented as a quintuple $(S, A, P, R, \gamma)$. Specifically, the state space $S$ represents all possible states in the environment; the action space $A$ represents all possible actions the agent can take; the state transition probability $P$ represents the probability of transitioning from state $s$ to another state; the reward function $R$ defines the reward received when taking action $a$ in state $s$; and the discount factor $\gamma \in [0, 1]$ balances the importance of current and future rewards.

In the context of RL, robot path planning involves the agent selecting actions based on the current state $s_t$ and sending them to the environment as the robot moves through it (Sun et al., 2021). The environment then transitions from state $s_t$ to the next state $s_{t+1}$ and provides a reward. The robot updates its policy based on the reward, continually interacting with the environment until it learns the optimal policy that maximizes cumulative rewards. This cumulative reward approach means the agent does not only choose the immediate best reward but considers the maximum reward over future scenarios. For example, the reward at time step $t$ is $r_t$, and the corresponding cumulative reward $G_t$ is:

$$G_t = \sum_{k=0}^{T} \gamma^k r_{t+k}, \tag{8}$$

where $r_{t+k}$ is the reward received at time step $t + k$. The parameter $T$ represents the terminal state, indicating the end of an episode.

To evaluate the value of states and actions, value functions are introduced, including the state value function $V^\pi(s_t)$ and the state–action value function $Q^\pi(s_t, a_t)$. The state value function $V^\pi(s_t)$, commonly known as the V value, represents the expected cumulative reward starting from state $s_t$ under policy $\pi$. The state–action value function $Q^\pi(s_t, a_t)$, commonly known as the Q value, represents the expected cumulative reward when taking action $a_t$ in state $s_t$ under policy $\pi$. The definitions of these functions are:

$$V^\pi(s_t) = \mathbb{E}[G_t|s_t] = \mathbb{E}[r_{t+1} + \gamma V^\pi(s_{t+1})|s_t], \tag{9}$$

$$Q^\pi(s_t, a_t) = \mathbb{E}[G_t|s_t, a_t] = \mathbb{E}[r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1})|s_t, a_t]. \tag{10}$$

By maximizing these value functions, RL can derive the optimal policy. However, in robot path planning, the vast state and action spaces make it challenging to accurately represent and calculate states and actions (Konar et al., 2013). To address this, DRL combines neural networks with RL to approximate value functions, allowing the algorithm to automatically extract features, enabling it to handle complex, high-dimensional environments effectively.

## 4. Methods

DRL addresses challenges posed by high-dimensional state and action spaces, thereby enhancing the generalization ability of RL. In this section, we will explain how the SAC algorithm is integrated with the DWA method for path planning, as well as the use of the tile coding method and our designed non-sparse reward function.

### 4.1. SAC-based DWA algorithm for path planning

The SAC algorithm (Haarnoja et al., 2018a) is a DRL method that combines experience learning with entropy maximization. Unlike conventional RL algorithms that primarily focus on maximizing the value function, which often inclines toward exploitation over exploration, SAC incorporates entropy into policy optimization, enabling better exploration of the environment and helping to avoid local optima. By maximizing policy entropy, SAC enhances exploration while integrating the actor–critic (AC) framework. This approach achieves a more balanced trade-off between exploration and exploitation, resulting in a more robust and efficient policy.

Subsequently, Haarnoja et al. (2018b) introduced an automatic entropy adjustment mechanism into SAC. This enhancement further fine-tunes the balance between exploration and exploitation, allowing SAC to dynamically adapt its exploration strategy based on the learning process, thereby improving stability and efficiency. Additionally, SAC incorporates an experience replay mechanism, which boosts sample efficiency and algorithm stability. The experience replay mechanism stores past experiences and randomly samples them during training, breaking data correlations and enhancing performance (Lin, 1992).

In our method, we integrate SAC with DWA, leveraging SAC's entropy maximization strategy, automatic entropy adjustment mechanism, and experience replay to enhance the path planning capabilities. This integration allows SAC to adjust DWA parameters in real-time within dynamic environments, providing a more reliable and efficient path planning strategy.

To fully understand the role of SAC in our integrated approach, it is important to revisit its foundational concepts. The following subsections discuss key aspects of maximum entropy, soft policy iteration, and automatic entropy adjustment, which are central to SAC's effectiveness in dynamic path planning.

#### 4.1.1. Maximum entropy

In the SAC algorithm, the objective function includes maximizing both the expected cumulative reward and the policy entropy. The maximization of policy entropy is usually achieved through entropy regularization. Higher policy entropy leads to more exploration by the agent. By including an entropy term in the objective function, SAC maintains a degree of randomness in action selection during policy optimization. Specifically, the policy $\pi(a|s)$ describes the probability of taking action $a$ in state $s$, and $\pi(\cdot|s)$ represents the action distribution given state $s$. Policy entropy measures the uncertainty of this probability distribution and is defined as:

$$\mathcal{H}(\pi(\cdot|s)) = -\sum_a \pi(a|s) \log \pi(a|s). \tag{11}$$

By maximizing the sum of the cumulative reward and policy entropy, SAC increases the randomness of the policy while learning it. High-entropy policies make broader action choices in different states, promoting exploration and avoiding premature convergence to local optima. The objective function of SAC is:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\Big[\sum_{t=0}^{T} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))\Big], \tag{12}$$

where, $\tau$ represents a trajectory (a sequence of states, actions, and rewards), $\tau \sim \pi$ indicates a trajectory sampled from the current policy $\pi$, $T$ represents the time steps of interaction between the robot and the environment, and $\alpha$ is the temperature parameter that balances reward and entropy. $\mathbb{E}$ represents the expected cumulative reward, $r(s_t, a_t)$ represents the immediate reward for choosing action $a_t$ in state $s_t$, and $\mathcal{H}(\pi(\cdot|s_t))$ represents the entropy of the current policy in state $s_t$. In SAC, maximizing entropy is a fundamental aspect of policy optimization, facilitating broader exploration by the agent and thereby reducing the risk of getting stuck in local optima.

#### 4.1.2. Soft policy iteration

In the SAC algorithm, the soft state–action value function is defined as:

$$\mathcal{T}^\pi Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}\sim p}[V(s_{t+1})], \tag{13}$$

where $\mathcal{T}^{\pi}$ denotes the Bellman backup operator used in soft policy iteration, applying the expected reward and future state values to update the current state–action value. $\mathbb{E}_{s_{t+1}\sim p}$ denotes the expectation over the next state, where $s_{t+1}$ is obtained according to the environment's state transition probability distribution $p$. The soft state value function is defined as:

$$V(s_t) = \mathbb{E}_{a_t\sim\pi}[Q(s_t, a_t) - \alpha \log \pi(a_t \mid s_t)], \tag{14}$$

where $a_t \sim \pi$ indicates that $a_t$ is sampled from the policy $\pi$ in state $s_t$.

Soft policy iteration aims for convergence to the soft Q-values, with each iteration defined as $Q^{k+1} = \mathcal{T}^{\pi}Q^{k}$, progressively refining the policy estimates. Here, $k$ represents the iteration number, and with each iteration, the soft Q-value is updated, gradually converging to a stable value.

Following the Q-value update, the algorithm progresses to the policy improvement phase, where adjustments are based on the refined soft Q-value distribution:

$$\pi_{\text{new}} = \arg\min_{\pi'\in\Pi} D_{KL}(\pi'(\cdot \mid s_t) \parallel \frac{\exp(Q^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)}), \tag{15}$$

where $D_{KL}$ represents the Kullback–Leibler (KL) divergence, measuring the difference between two distributions. $\pi_{\text{new}}$ is the updated policy, $\pi_{\text{old}}$ is the current policy, and $\Pi$ represents the set of all feasible policies. $\pi'$ denotes a candidate policy within the set $\Pi$. Additionally, $Q^{\pi_{\text{old}}}(s_t, \cdot)$ is the soft Q-value distribution based on state $s_t$. The normalization factor $Z^{\pi_{\text{old}}}(s_t)$ ensures that the Q-values are proportionately adjusted across all possible actions, maintaining a balanced exploration–exploitation trade-off. This formula seeks to derive a new policy that closely aligns with the current policy's soft Q-value distribution by minimizing the KL divergence, guiding policy improvement. Through iteration, the overall policy is continuously optimized.

In each iteration, policy evaluation is followed by policy improvement. After evaluating the current policy, the policy value is updated and used to guide the next policy improvement step. This iterative process allows SAC to balance exploration and exploitation during learning, effectively converging to the optimal policy.

#### 4.1.3. Automatic entropy adjustment SAC

The automatic entropy adjustment mechanism enables the SAC algorithm to adaptively balance exploration and exploitation, which is beneficial in dynamic environments. This adaptive adjustment is crucial in dynamic environments, where changing states and varying actions require different levels of exploration. Through the automatic entropy adjustment mechanism, the SAC algorithm can dynamically adapt to these changes.

When the SAC algorithm is combined with DWA for path planning, the environment includes a continuous state space. Additionally, actions correspond to the weights in DWA, which include target heading angle deviation, distance to the nearest obstacle, and velocity weights. These weights are all within the range [0,1], and the action is represented as $a = (w_{head}, w_{dist}, w_{vel})$. Effective management of these continuous spaces necessitates the parameterization of both the policy network and the state–action value function network, using these parameterized networks to approximate and optimize the policy and value function. In subsequent discussions, the state–action value function network will be referred to as the Q-value network.

To reduce bias in the Q-value function estimation during parameter updates, the SAC algorithm typically uses double Q-networks to estimate the state–action value function. Therefore, this paper adopts the same network structure as Shi et al. (2020), which includes a policy network $\pi_{\phi}$, dual Q-value networks $Q_{\theta_{i=1,2}}$, and two corresponding target Q-value networks $\hat{Q}_{\theta'_{i=1,2}}$.

The structure of the SAC network is illustrated in Fig. 2. This network structure not only accurately estimates the action value but also reduces estimation bias through the soft update mechanism of the target Q-value networks, improving the algorithm's stability and learning efficiency. The objective function for the dual Q-value networks is defined as:

$$J_Q(\theta_i) = \mathbb{E}_{(s_t,a_t)\sim D}\left[\frac{1}{2}\left(Q_{\theta_{i=1,2}}(s_t, a_t) - \hat{Q}(s_t, a_t)\right)^2\right], \tag{16}$$

where $D$ is the replay buffer storing samples of states, actions, rewards, and next states sampled from the environment, $Q_{\theta_{i=1,2}}(s_t, a_t)$ are the Q-value estimates from the dual Q-value networks for state $s_t$ and action $a_t$. The target value $\hat{Q}(s_t, a_t)$ is defined as:

$$\hat{Q}(s_t, a_t) = r_t + \gamma\left(\min_{i=1,2} Q_{\theta'_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\phi}(a_{t+1} \mid s_{t+1})\right), \tag{17}$$

where $r_t$ is the immediate reward, $s_{t+1}$ is the next state obtained from the state transition probability distribution $p(s_{t+1} \mid s_t, a_t)$, and $a_{t+1}$ is the action chosen by the policy in the next state $s_{t+1}$. $\gamma$ is the discount factor, $\theta'_i$ are the parameters of the target Q-value networks, and $\min_{i=1,2} Q_{\theta'_i}(s_{t+1}, a_{t+1})$selects the smaller Q-value from the two target Q-networks to reduce estimation error, stabilizing the update of $\log \pi_{\phi}$.

Using the objective function of the Q-value networks and the target Q-value networks, the gradient of $J_Q(\theta_i)$ with respect to the parameters $\theta_i$ is calculated as:

$$\nabla_{\theta_i} J_Q(\theta_i) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim D}\left[(Q_{\theta_i}(s_t, a_t) - \hat{Q}_{\theta_i}(s_t, a_t))\nabla_{\theta_i} Q_{\theta_i}(s_t, a_t)\right]. \tag{18}$$

Additionally, the parameter update formula for the target Q-value networks is:

$$\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i, \quad \tau \in [0, 1], \tag{19}$$

where $\tau$ is the soft update parameter.

Given the continuous nature of the action space defined in this paper, and since the SAC algorithm outputs the mean and standard deviation of a Gaussian distribution for continuous action spaces, the action sampling process from the Gaussian distribution is non-differentiable (Liu et al., 2023). To address this, the reparameterization trick is applied. This technique involves sampling from a standard normal distribution $N(0, 1)$ and transforming it using the mean and standard deviation produced by the policy network:

$$a_t = \mu(s_t \mid \theta) + \sigma(s_t \mid \theta) \odot \varepsilon_t = f_{\theta}(\varepsilon_t; s_t), \tag{20}$$

where $\mu(s_t \mid \theta)$ and $\sigma(s_t \mid \theta)$ are the mean and standard deviation of the action, and $\varepsilon_t$ is a noise variable sampled from the normal distribution $N(0, 1)$. $\odot$ denotes element-wise multiplication. Substituting this into the objective function and considering the dual Q-value functions, the rewritten soft Q-value objective function is:

$$J_Q(\theta_i) = \mathbb{E}_{s_t\sim D,\varepsilon_t\sim N(0,1)}\left[\frac{1}{2}\left(Q_{\theta_{i=1,2}}(f_{\theta}(\varepsilon_t; s_t) \mid s_t) - \min_{i=1,2} Q_{\theta'_i}(s_t, f_{\theta}(\varepsilon_t; s_t))\right)^2\right]. \tag{21}$$

In the SAC algorithm, in addition to considering continuous action spaces, the selection of the entropy parameter $\alpha$, or the temperature parameter, is also very important. Different levels of entropy are needed in different states. In states where the optimal action is uncertain, $\alpha$ should be increased to encourage exploration. Conversely, in states where the optimal action is more certain, $\alpha$ should be decreased to enhance policy certainty. In conventional SAC algorithms, the temperature parameter is a fixed hyperparameter. However, the optimal temperature parameter may vary in different environments. Therefore, to adaptively adjust the temperature parameter and balance exploration and exploitation, SAC introduces an automatic entropy adjustment mechanism. Specifically, the automatic entropy adjustment introduces a target entropy $\mathcal{H}_{target}$, allowing $\alpha$ to approach this target value. The objective function of $\alpha$ is:

$$J(\alpha) = \mathbb{E}_{s_t\sim D,a_t\sim\pi_{\phi}}[-\alpha(\log \pi_{\phi}(s_t, a_t) + \mathcal{H}_{target})]. \tag{22}$$

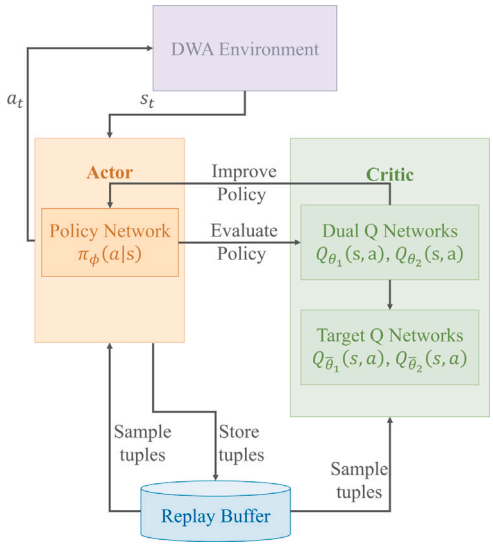


**Fig. 2.** Network architecture of the SAC algorithm.

By minimizing this objective function, $\alpha$ can approach the target entropy. The specific update rule is:

$$\alpha \leftarrow \alpha - \eta_\alpha \nabla_\alpha J(\alpha), \tag{23}$$

where $\eta_\alpha$ is the learning rate, and the gradient $\nabla_\alpha J(\alpha)$ is:

$$\nabla_\alpha J(\alpha) = -\mathbb{E}_{s_t \sim D, a_t \sim \pi_\phi}[\log \pi_\phi(a_t \mid s_t) + \mathcal{H}_{target}]. \tag{24}$$

The automatic entropy adjustment mechanism ensures that policy entropy remains close to the target entropy, thereby automating the entropy adjustment process and maintaining exploration capability in uncertain or diverse action space environments. This feature enhances SAC's ability to adapt to varying conditions and diverse action spaces, which is important for dynamic path planning in complex environments requiring broader exploration to identify effective strategies.

In this paper, we integrate the SAC algorithm, which incorporates an automatic entropy adjustment mechanism, with DWA for local path planning in dynamic environments. This integration allows SAC to adapt to environmental changes during the robot's path planning process. The process begins by setting the robot at its start point within a dynamic environment. The initial state, captured from the DWA environment, serves as the starting point for SAC's operations. Central to this process is SAC's policy network, which plays a crucial role by calculating the action weights: $w_{head}$ for target heading angle deviation, $w_{dist}$ for distance to the nearest obstacle, and $w_{vel}$ for velocity—based on the current state. These weights, derived from the policy network's learning capabilities, optimize decision-making and directly influence subsequent DWA operations.

After the policy decision, the DWA takes over to perform velocity sampling, trajectory prediction, and trajectory evaluation, utilizing the weights provided by the SAC. This evaluation process tests the efficacy of the chosen actions under real conditions and selects the optimal combination of linear and angular velocities for the robot to adopt.

As the robot performs path planning in the environment, it continually collects sensory data and updates its state. The collected state information is then fed into the reward calculation module to generate a reward. The updated state and reward are subsequently provided to the SAC algorithm. The SAC's dual Q-value networks play a pivotal role by estimating potential rewards for specific actions at given states, thus creating a feedback loop that refines the policy network's outputs. To further enhance the system's robustness in exploring the action space, the SAC algorithm incorporates an automatic entropy adjustment mechanism. This feature dynamically adjusts entropy levels, ensuring that exploration does not stagnate and that the algorithm remains adaptive to the inherent uncertainties present in dynamic environments.

This iterative process of action selection, evaluation, and network adjustment continues throughout the training phase of the robot within its environment. The continuous feedback loop enables the SAC algorithm to adaptively tune the action weights in response to real-time environmental changes, thereby enhancing the effectiveness and responsiveness of the path planning. This approach ensures efficient and real-time path planning.

### 4.2. Using tile coding to improve state space

By integrating the SAC algorithm with an automatic entropy adjustment mechanism and the DWA algorithm, we precisely define the action space. Building on this foundation, designing an appropriate state space is crucial for improving the effectiveness of path planning in dynamic environments. This paper proposes a method that applies tile coding to state feature representation, aiming to address potential issues of improper generalization between input states.

All DRL algorithms utilize state information as feature vectors, which are input into the network. Actions are then selected based on the policy for the current state, rewards are obtained, and the policy is further optimized through cumulative rewards. Path planning in dynamic environments is more complex than in static environments. Therefore, to better adapt the robot's path planning to dynamic environments, it is essential to fully consider their dynamic characteristics when defining states (Chen et al., 2022).

In this paper, the original state consists of two elements: the robot's motion information $s_{rob}$ and the detected environmental information $s_{det}$. $s_{det}$ mainly includes information about the obstacles around the robot at the current moment, helping the robot perceive changes in its surroundings. Fig. 3 illustrates how the robot detects the environment using sensors in a simulated environment. The robot is at the center, and the circle centered on the robot represents its detection range. The radius of the circle, $d_{det}$, indicates the robot's maximum detection distance. Through its sensors, the robot can obtain specific information about all obstacles within this detection range, including their positions, velocities, movement directions, and distances from the robot. The detected environmental information $s_{det}$ is specifically defined as:

$$s_{det} = [x_o, y_o, v_{ox}, v_{oy}, d_{r-o}, \phi_{diff}], \tag{25}$$

here, $x_o$ and $y_o$ represent the coordinates of the nearest obstacle relative to the robot in the two-dimensional plane. $v_{ox}$ and $v_{oy}$ represent the velocities of the nearest obstacle along the $x$ and $y$ axes, respectively. These velocities can be used to calculate the obstacle's movement direction. $d_{r-o}$ represents the distance between the robot and the nearest obstacle. $\phi_{diff}$ represents the absolute value of the angular difference between the robot and the nearest obstacle. Additionally, during DRL training, the state dimensions need to remain consistent. If there are no obstacles within the detection range, the coordinates of the nearest obstacle are set to the top-left corner of the environment, and the velocities on both the $x$ and $y$ axes are set to 0 to maintain dimensional consistency. The robot's motion information, $s_{rob}$, primarily includes data related to the robot's movement at the current moment, which helps the robot understand and control its motion state. Specifically, the robot's motion information $s_{rob}$ is defined as:

$$s_{rob} = [x_r, y_r, v_r, v_w, \phi_r, d_{ending}, \phi_{ending}], \tag{26}$$

where $x_r$ and $y_r$ represent the current coordinates of the robot in a two-dimensional plane, and $v_r$ and $v_w$ denote the robot's linear and angular velocities, respectively. Additionally, $\phi_r$ represents the robot's current heading angle, $d_{ending}$ denotes the distance between the robot's center and the end point, and $\phi_{ending}$ represents the angle between the robot's heading and the direction to the end point.

Although DRL improves RL's generalization ability using neural networks, improper generalization between input states can lead to

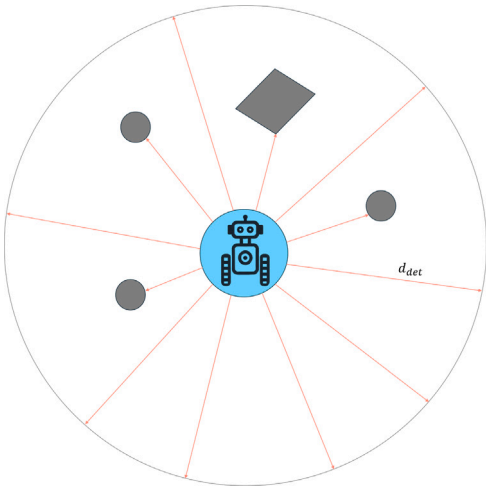


**Fig. 3.** Robot environmental sensing in simulated environments.

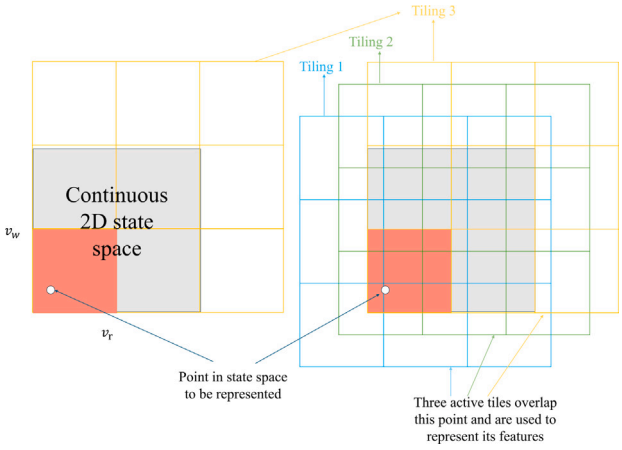


**Fig. 4.** Integrated structure of SAC with DWA for path planning.

difficulties in DRL training. Therefore, when defining the robot's state, it is important to focus on the significant features within the original state to more precisely capture the robot's dynamic characteristics and reduce improper generalization. To address this issue, we introduced tile coding to supplement the representation of the robot's linear and angular velocities in the original state, enabling the robot to better adapt to changes in dynamic environments and reducing the risk of overgeneralization in the neural network.

Tile coding discretizes continuous spaces by overlaying them with multiple offset grids, known as 'tilings'. Each grid divides the state space into small squares, called tiles, with each tile having a unique identifier. Specifically, as shown in Fig. 4, we first take the continuous space of linear and angular velocities as input and cover this space with multiple tilings to ensure that every point in the continuous space falls within each tiling. The linear and angular velocities under different states can be represented by a single point in this space. Each tiling is then evenly divided into $m \times n$ tiles, where $m$ and $n$ represent the number of divisions along the linear and angular velocity axes, respectively. Therefore, when the state point is updated — i.e., when either the linear or angular velocity changes — the state point will activate multiple tiles simultaneously. For example, in Fig. 4, the state is divided into three overlapping tilings, corresponding to three activated tiles. In practical applications, the values of $m$, $n$, and the number of tilings can be defined according to the user's specific needs.

Tile coding mitigates improper generalization through multiple overlapping tilings (Waskow and Bazzan, 2010). This overlapping encoding ensures that each point in the state space, formed by the linear and angular velocities, is covered by multiple tiles, reflecting feature changes across multiple dimensions. In the actual implementation, we input the linear $v_r$ and angular velocities $v_w$ from the state into the tile coding process, generating discrete feature representations through multiple tilings, and then combine these with the original state to form a new state representation. Specifically, the output of tile coding is the set of indices of the activated tiles, which, combined with the original state, forms the new state that is input into the SAC algorithm. Consequently, the new state not only includes the robot's motion information $s_{rob}$ and the detected environmental information $s_{det}$ but also integrates the state representation $s_{tile}$ generated through tile coding. Thus, the new state is defined as $s_{new} = \{s_{rob}, s_{det}, s_{tile}\}$.

Furthermore, as each dimension of $s_{tile}$ corresponds to tile indices, which are typically larger than those in $s_{rob}$ and $s_{det}$, all dimensions of the state in $s_{new}$ are normalized. Specifically, the normalization of $s_{tile}$ is achieved by dividing the tile indices in each dimension by the maximum tile index of the corresponding tiling, ensuring that each dimension of $s_{tile}$ is normalized to the range [0,1]. Similarly, the values of all dimensions in $s_{rob}$ and $s_{det}$ are normalized to the range [0,1] based on their respective value ranges to ensure that all features are compared on the same scale.

### 4.3. Reward function

In DRL algorithms, the agent takes actions at each timestep and receives corresponding rewards based on those actions. The reward function, which provides feedback based on the agent's actions, directly influences the learning process and decision-making. In DRL-based path planning, conventional sparse reward functions typically do not provide rewards in most states, only giving positive or negative rewards upon reaching specific goals or encountering special events, such as arriving at the end point or colliding with obstacles. The sparsity of feedback signals can hinder the agent's early training by making it difficult to provide local guidance, which leads to blind exploration, low sample efficiency, and slow convergence. To address this issue, we designed a non-sparse reward function that integrates TTC with three evaluative factors from the DWA algorithm's evaluation function: target heading angle deviation, distance to the nearest obstacle, and velocity.

The designed reward function consists of three components: a timestep-based regular reward $R_n$, a reward for reaching the end point $r_{end}$, and a penalty for colliding with obstacles $r_{col}$. The values of $r_{end}$ and $r_{col}$ are set to 100 and −100, respectively. The regular reward $R_n$ is defined as the reward that the agent receives at each time step, except when the agent either reaches the end point or collides with an obstacle. $R_n$ incorporates three key factors and is defined as:

$$R_n = r_{head} + r_{ttc} + r_{vel} + r_{step}, \tag{27}$$

where $r_{head}$ represents the target heading angle deviation factor in the DWA evaluation function. Its value is inversely proportional to the target angle deviation; the smaller the target angle deviation, the closer the robot is to moving directly toward the desired end point. Therefore, the smaller the target angle deviation, the greater the reward value. Thus, $r_{head}$ is defined as follows:

$$r_{head} = \frac{\pi - \|\phi_{ending}\|}{\pi}, \tag{28}$$

where $\phi_{ending}$ represents the angle between the robot's heading and the direction to the end point.

$r_{ttc}$ represents the distance to the nearest obstacle, which is another crucial factor in the DWA evaluation function in addition to the heading deviation. We use a piecewise function based on TTC to calculate the reward value. TTC estimates the time remaining before the robot collides with an obstacle, given its current motion state (Saffarzadeh et al., 2013). The calculation of TTC requires considering the relative

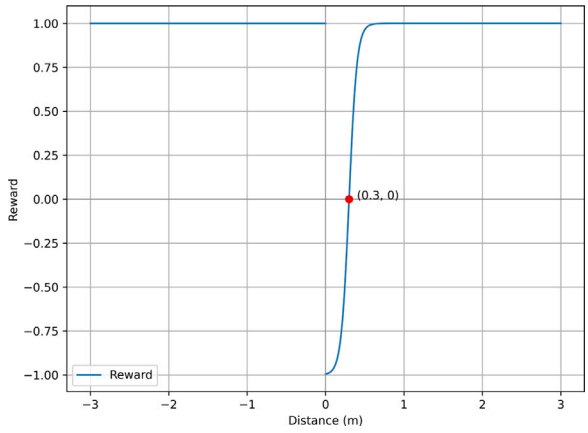


**Fig. 5.** Piecewise function for the TTC-Based reward.

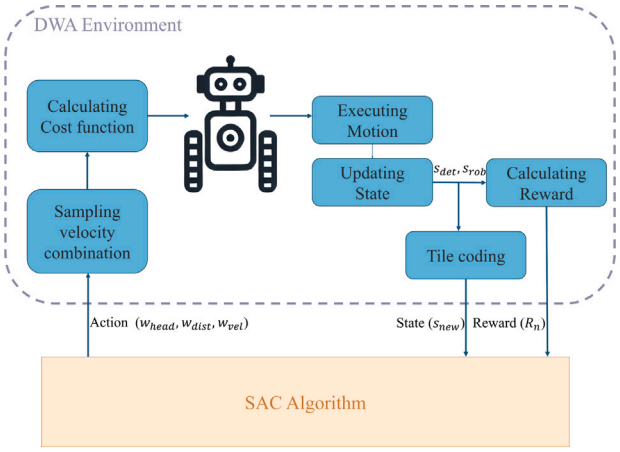


**Fig. 6.** Integrated structure of SAC with DWA for path planning.

direction and velocity between the robot and the obstacle. The formula is given by:

$$\mathrm{TTC} = \frac{-(\vec{p_{rel}} \cdot \vec{v_{rel}})}{|\vec{v_{rel}}|^2}, \tag{29}$$

where $\vec{p_{rel}}$ denotes the relative position between the robot and the nearest obstacle, calculated as the difference between their respective coordinates: $\vec{p_{rel}} = (x_r - x_o, y_r - y_o)$. $\vec{v_{rel}}$ represents the relative velocity between the robot and the obstacle, given by the difference between their respective velocities: $\vec{v_{rel}} = (v_r \cdot \cos(\phi) - v_{ox}, v_r \cdot \sin(\phi) - v_{oy})$. The coordinates and velocities are consistent with the values in the state information $s_{new}$. The TTC value can be positive or negative. A positive TTC value indicates that the robot will collide with the obstacle after that duration, while a negative TTC value suggests that no collision will occur. To reflect the collision risk in these scenarios, we designed a piecewise function to assign corresponding rewards or penalties. The value range of this piecewise function is set between $[-1, 1]$, with rewards assigned based on the TTC value. For $x \geq 0$, we introduced a modified sigmoid function to smoothly distribute reward values. The specific piecewise function is:

$$r_{ttc} = \begin{cases} 1, & x < 0 \\ \frac{2}{1+e^{-20(x-0.3)}} - 1, & x \geq 0. \end{cases} \tag{30}$$

In the piecewise function, a TTC value less than zero ($x < 0$), indicating no impending collision, yields a constant reward of 1. Conversely, for a positive TTC value ($x \geq 0$), predicting a collision within $x$ seconds, a modified sigmoid function calculates the reward value. This approach rapidly decreases the reward when TTC is small and provides a positive reward when the collision time is relatively safe. The piecewise function is illustrated in Fig. 5.

$r_{vel}$ represents the velocity factor in the DWA evaluation function. Its value is proportional to the robot's current velocity, meaning that the faster the velocity, the higher the reward. The value of $r_{vel}$ is calculated as follows:

$$r_{vel} = \frac{v_r}{v_{max}}, \tag{31}$$

where $v_{max}$ is the robot's maximum velocity.

$r_{step}$ is an additional time step reward implemented to encourage the agent to reach the end point as quickly as possible. By assigning a negative reward of −3 at each time step, the agent is incentivized to minimize the time taken to reach the goal. Additionally, the results of $r_{head}$, $r_{ttc}$, and $r_{vel}$ are all normalized values. Ultimately, the agent's reward at each time step is the sum of these four components.

The complete integration of the SAC with an automatic entropy adjustment mechanism, tile coding, and the designed reward function is illustrated in Fig. 6. During the robot's path planning process, it continuously gathers sensory information represented as $s_{det}$ and $s_{rob}$. This information is processed through tile coding and merged into a new state representation $s_{new}$ at each step. While $s_{det}$ and $s_{rob}$ are used in the reward calculation module to generate the reward $R_n$, the combined state $s_{new}$, along with the reward $R_n$, is subsequently fed into the SAC algorithm. The SAC model then determines the optimal DWA weighting parameters $w_{head}$, $w_{dist}$, and $w_{vel}$ to guide the robot's path planning. In each episode, this process repeats as the SAC algorithm continuously trains and updates the policy. The robot iteratively updates its state and adjusts the DWA parameters until it either reaches the end point or meets a termination condition.

## 5. Results

To validate the feasibility of the ASAC algorithm in dynamic environments, we conducted experiments in four simulated environments and performed a comparative analysis of its performance against other algorithms, including SAC, SAC*, PPO (Schulman et al., 2017), and DDPG (Lillicrap et al., 2015). All experiments were conducted on a Microsoft Windows 10 system equipped with an Intel i5 3.0 GHz CPU and 16 GB of RAM. All algorithms were implemented in Python and trained and tested in a simulation environment built using Pygame.

### 5.1. Experimental setup

To enhance the robustness and adaptability of the model, the training environment was designed with obstacles featuring random initial positions and velocities. Specifically, the obstacles' velocities ranged from 0 to 0.2 m/s along both the $x$-axis and $y$-axis, with randomly assigned movement directions, either positive or negative. The number of obstacles in the training environment was within the range [5, 8]. Additionally, the obstacles' positions were initialized randomly within the environment. However, to prevent immediate interference with the robot's starting state, their initial positions were constrained to be outside a 0.3-m radius from the starting point. When obstacles reach the boundaries of the training environment, they bounce off the walls.

The training environment measures 8 m × 6 m, corresponding to x-coordinates ranging from −4 to 4 and y-coordinates ranging from −3 to 3. The robot's starting and target positions are set at (−3.5, −2.5) and (3.5, 2.5), respectively. Both the robot and the obstacles have a radius of 0.1 m.

To ensure consistent performance comparisons across algorithms, all algorithms are trained with the same hyperparameter settings and neural network architecture. All randomization processes, including the movement direction, velocity, and positions of obstacles, used a

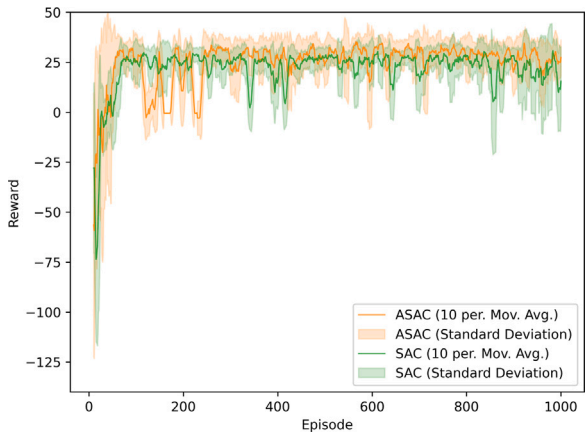


**Fig. 7.** Episode reward evolution for ASAC and SAC.

fixed random seed of 42. Each neural network contains three hidden layers with 64, 32, and 16 neurons, respectively. The detailed hyperparameter settings are listed in Table 1. In Table 1, parameters such as $alpha_{learning_{rate}}$, $target_{entropy}$, $train_{freq}$, $gradient_{steps}$, and $\tau$ are specific to ASAC, SAC*, and SAC, while PPO and DDPG use the other listed parameters. Additionally, the time step for each iteration during training is set to 0.1 s. When applying tile coding for state feature representation, the range for linear velocity is set to $[-0.2, 1.0]$, divided into 10 equal intervals. The range for angular velocity is set to $[-\pi, \pi]$, divided into 30 equal intervals, with 6 tilings applied.

### 5.2. Performance comparisons

We trained the ASAC, as well as SAC, SAC*, PPO, and DDPG algorithms, all of which are actor–critic-based DRL methods that optimize policies through continuous interaction with the environment. ASAC combines tile coding for feature representation with an automatic entropy adjustment mechanism. In contrast, SAC* incorporates only the automatic entropy adjustment, while SAC, PPO, and DDPG directly input the state into the model without these enhancements. The effectiveness of each algorithm's policy optimization was measured by the cumulative reward value in each episode, either upon reaching the maximum time step or the end point.

To compare the performance differences between ASAC and SAC, Fig. 7 illustrates the reward variations of these two algorithms over 1000 episodes. The horizontal axis represents the episode number, while the vertical axis indicates the reward value for each episode. Each line in the figure represents the moving average of the rewards, with each point calculated as the average over the previous 10 episodes. The shaded area indicates the standard deviation for each point in the moving average. While Fig. 7 focuses on the comparison between ASAC and SAC, Table 2 supplements this analysis by providing the average rewards of ASAC, SAC, SAC*, PPO, and DDPG over every 100 episodes. This offers a more comprehensive assessment of the performance trends across different algorithms.

As shown in Fig. 7 and Table 2, the rewards of both ASAC and SAC exhibit significant fluctuations during the first 200 episodes, with the standard deviations gradually stabilizing in subsequent episodes. Specifically, ASAC's average reward during episodes 0 to 100 was 8.94, with a standard deviation of 40.90. From Fig. 7, it can be observed that ASAC experienced substantial reward fluctuations between episodes 110 and 230 after approaching a peak, indicating that its automatic entropy adjustment mechanism is actively exploring a wider range of actions, thereby enhancing policy diversity and optimization potential. This corresponds with the larger standard deviation observed for ASAC during this period. In contrast, SAC shows a smaller and more stable standard deviation.

**Table 1**
Relevant hyperparameters for all algorithms.

| Description | Parameter | Value |
|---|---|---|
| Discount factor | gamma | 0.98 |
| $\alpha$ network learning rate | alpha_learning_rate | 3e−4 |
| Actor network learning rate | actor_learning_rate | 3e−4 |
| Critic network learning rate | critic_learning_rate | 3e−4 |
| Buffer size | buffer_size | 5e3 |
| Batch size | batch_size | 256 |
| Target entropy | target_entropy | −3 |
| Train frequency | train_freq | 1 |
| Number of gradient steps for each update | gradient_steps | 1 |
| Soft update parameters | tau | 5e−3 |
| Maximum number of time steps per episode | timesteps | 2e2 |
| Total training episodes | episodes | 1e3 |

**Table 2**
Average rewards for each algorithm across episode ranges.

| Episode range | PPO | DDPG | SAC | SAC* | ASAC |
|---|---|---|---|---|---|
| 0–100 | −4.09 | 1.82 | 3.85 | −10.45 | **8.94** |
| 100–200 | 15.99 | 11.83 | **25.22** | 10.71 | 12.68 |
| 200–300 | 18.85 | 11.83 | **24.98** | 22.09 | 23.23 |
| 300–400 | 19.87 | 11.83 | 21.97 | 27.45 | **27.64** |
| 400–500 | 19.39 | 11.83 | 24.03 | 26.68 | **26.89** |
| 500–600 | 17.03 | 11.83 | 25.14 | 26.24 | **28.44** |
| 600–700 | 20.69 | 11.83 | 24.27 | 28.04 | **29.95** |
| 700–800 | 19.76 | 11.83 | 25.45 | 26.92 | **29.00** |
| 800–900 | 21.78 | 11.83 | 22.87 | 26.02 | **28.10** |
| 900–1000 | 18.13 | 11.83 | 20.01 | 26.86 | **27.06** |

After the 200th episode, ASAC's average reward continued to increase, with the standard deviation stabilizing around 10. During the final 100 episodes, from 900 to 1000, ASAC achieved an average reward of 27.06, with a standard deviation of 10.24.

Additionally, we observed that despite the continuous changes in the position and velocity of obstacles, the standard deviation of the average reward for DDPG's average reward was consistently lower and varied less compared to other algorithms like PPO, ASAC, SAC*, and SAC. This phenomenon can be attributed to the fact that DDPG is a deterministic policy algorithm. Unlike stochastic policy algorithms such as ASAC, SAC, and PPO that introduce randomness in action selection, DDPG deterministically chooses actions given a specific state. This deterministic nature results in more consistent behavior across different training runs, leading to a lower standard deviation. In contrast, stochastic policy algorithms are inherently designed to explore a wider range of actions even under the same state, which contributes to the larger standard deviation in average rewards. Overall, during the training process, ASAC achieved the highest average rewards compared to PPO, SAC*, SAC, and DDPG.

### 5.3. Experimental results

To comprehensively evaluate the performance of ASAC, SAC, PPO, and DDPG, we designed four distinct experimental environments. It is important to note that the velocities of all obstacles mentioned below consist of two components: the first value represents the $x$-axis velocity component, and the second value represents the $y$-axis velocity component. If a velocity component is negative, it indicates that the direction of movement is opposite to the corresponding coordinate axis.

Fig. 8 illustrates these four experimental environments. The green dot represents the starting point, the orange dot indicates the target point, black represents static obstacles, and the blue dots with arrows represent dynamic obstacles and their directions of movement. Similar to the training environment, the starting and target positions for the robot in each environment were set at (−3.5, −2.5) and (3.5, 2.5), respectively. Additionally, the radii of the robot, dynamic obstacles, and non-polygonal obstacles are all set to 0.1 m. The robot was considered to have successfully reached the end point when its center

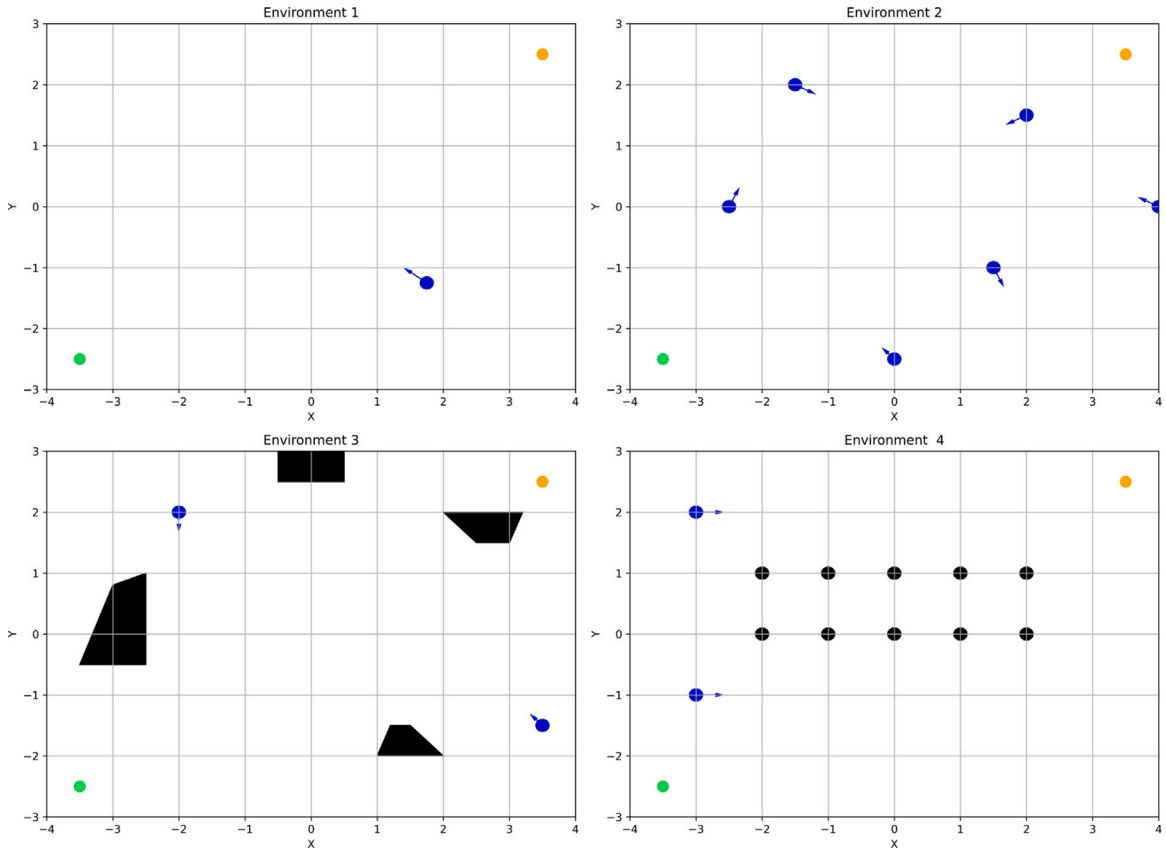


**Fig. 8.** Four experimental environments for path planning.

point was within 0.2 meters of the target. Environment 1 simulates a crossing scenario. In this environment, a dynamic obstacle moves along a path that intersects with the line connecting the robot's starting point (−3.5,−2.5) and its target point (3.5,2.5). The initial position of this dynamic obstacle is (1.75, −1.25), with a velocity of (−0.24, 0.18) m/s. Environment 2 simulates a multi-dynamic obstacle scenario. This environment contains six dynamic obstacles with initial positions at (−2.5, 0), (−1.5, 2), (−1, −2), (0.5, 1.5), (1.5, −1), and (2.5, 0). The corresponding velocities of these obstacles are (0.1, 0.2), (0.2, −0.1), (−0.1, 0.2), (−0.2, −0.1), (0.1, −0.2), and (0.2, 0.1) m/s.

Environment 3 simulates a mixed obstacle scenario, containing two dynamic obstacles and four static polygonal obstacles. The initial positions of the dynamic obstacles are (3.5, −1.5) and (−2, 2), with velocities of (−0.1, 0.1) and (0, −0.2) m/s, respectively. The vertices of the four static polygonal obstacles are as follows: [(−3.5, −0.5), (−3, 0.8), (−2.5, 1), (−2.5, −0.5)], [(1, −2), (2, −2), (1.5, −1.5), (1.2, −1.5)], [(2.5, 1.5), (3, 1.5), (3.2, 2), (2, 2)], and [(−0.5, 3), (0.5, 3), (0.5, 2.5), (−0.5, 2.5)].

Finally, Environment 4 simulates a crowded scenario. This environment contains ten static obstacles and two dynamic obstacles. The positions of the static obstacles are: (−2, 0), (−1, 0), (0, 0), (2, 0), (1, 0), (−2, 1), (−1, 1), (0, 1), (2, 1), and (1, 1). The initial positions of the dynamic obstacles are (−3, −1) and (−3, 2), both with velocities of (0.3, 0) m/s.

These environments allow for the evaluation of each algorithm's capability in obstacle avoidance and path planning under different scenarios. Compared to the training environment, the obstacles in Environment 1 move at higher velocities, and the crossing scenario increases the difficulty of path planning. The experimental results for the four algorithms in Environment 1 are shown in Fig. 9(a)–(d). Fig. 9(a) illustrates the path planned by the ASAC algorithm, where the robot successfully avoids the dynamic obstacles and reaches the target location. In Fig. 9(b), a comparison of the trajectories planned by the four algorithms indicates that while each algorithm successfully reaches the end point, there are differences in terms of path distance and obstacle avoidance performance. Fig. 9(c) presents the cumulative heading changes over time during the path planning process for the four algorithms, with the robot's arrival time and cumulative heading change at the end point indicated. The results indicate that the ASAC algorithm achieves the smallest cumulative heading change and the shortest arrival time, followed by SAC*, SAC, PPO, and DDPG. Fig. 9(d) shows the distance between the robot and the nearest obstacle during local path planning, with dots indicating the closest distances encountered along each algorithm's path. The results indicate that although the planned paths differ, all algorithms successfully avoid obstacles and reach the end point safely. Therefore, in Environment 1, the ASAC algorithm outperforms the other three algorithms in terms of smoother and more efficient path planning.

Environment 2 is the scenario most similar to the training environment. The experimental results in Environment 2 are shown in Fig. 10(a)–(d). Fig. 10(a) illustrates the path planned by the ASAC algorithm, where the robot again successfully avoids all dynamic obstacles and reaches the target location. The trajectories in Fig. 10(b) indicate that all algorithms successfully reach the end point. Fig. 10(c) presents the cumulative heading changes for the four algorithms. The ASAC algorithm achieves the smallest cumulative heading change, with a significant difference compared to the other algorithms. Specifically, the cumulative heading change for ASAC is 0.77 rads, while those for SAC, SAC*, PPO, and DDPG are 2.23 rads, 2.32 rads, 2.53 rads, and 2.75 rads, respectively, showing a sequential increase. Fig. 10(d) shows the distance between the robot and the nearest obstacle, with results similarly indicating that each algorithm successfully avoids obstacles and reaches the end point safely. Compared to Environment 1, the quality advantage of ASAC's planned path is even more apparent in Environment 2.

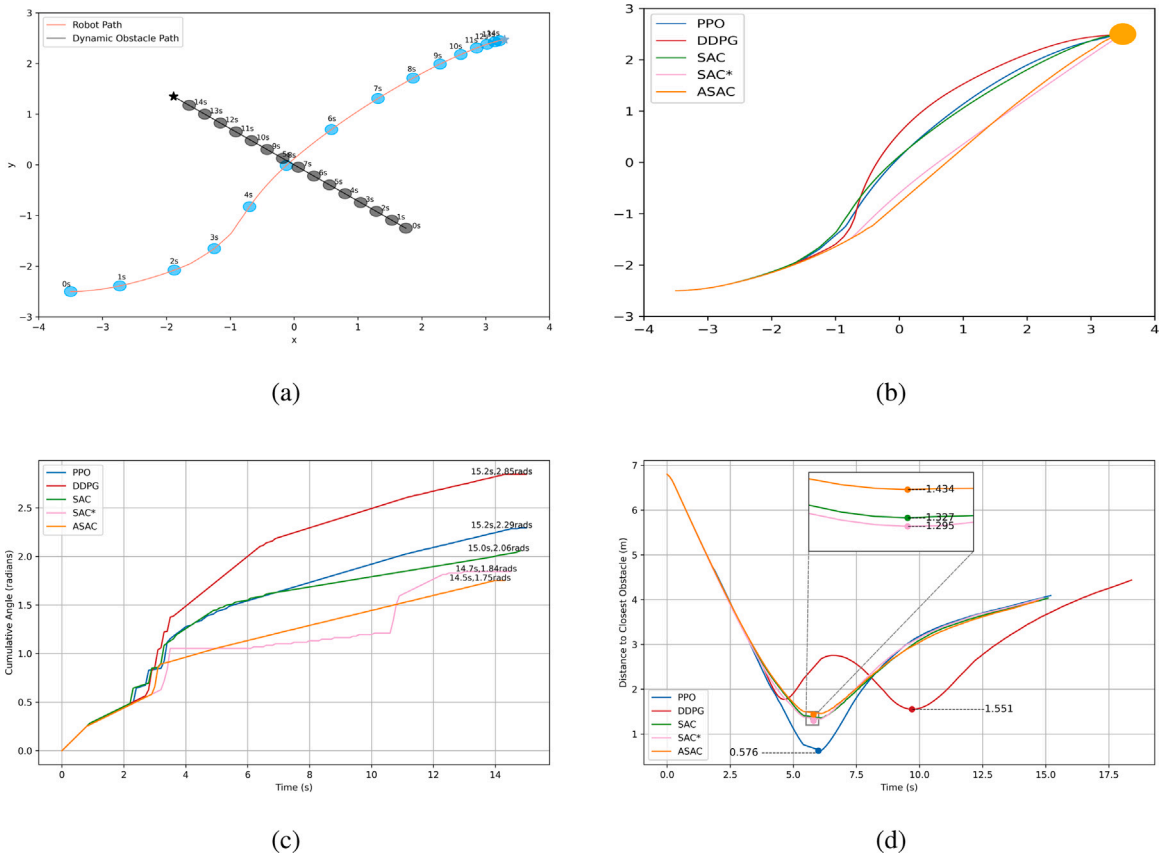


**Fig. 9.** Collision avoidance results and analysis in environment 1.

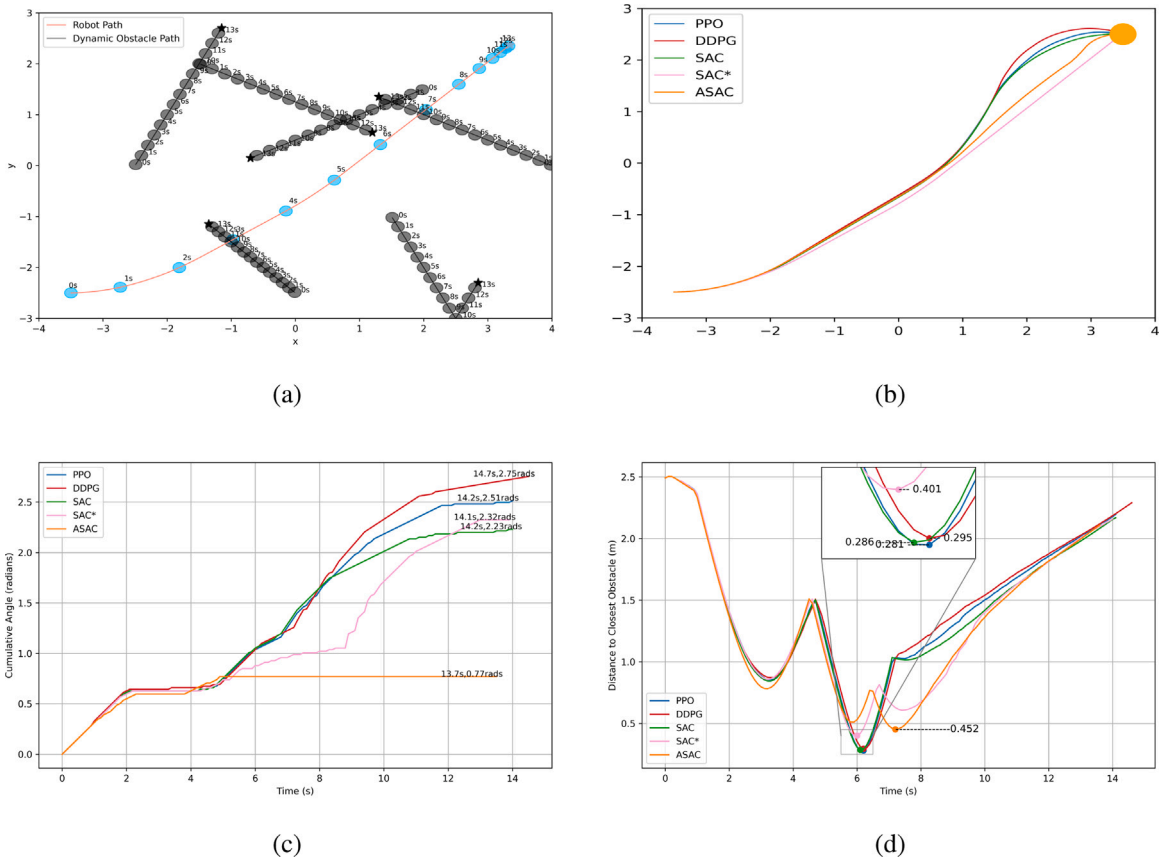


**Fig. 10.** Collision avoidance results and analysis in environment 2.

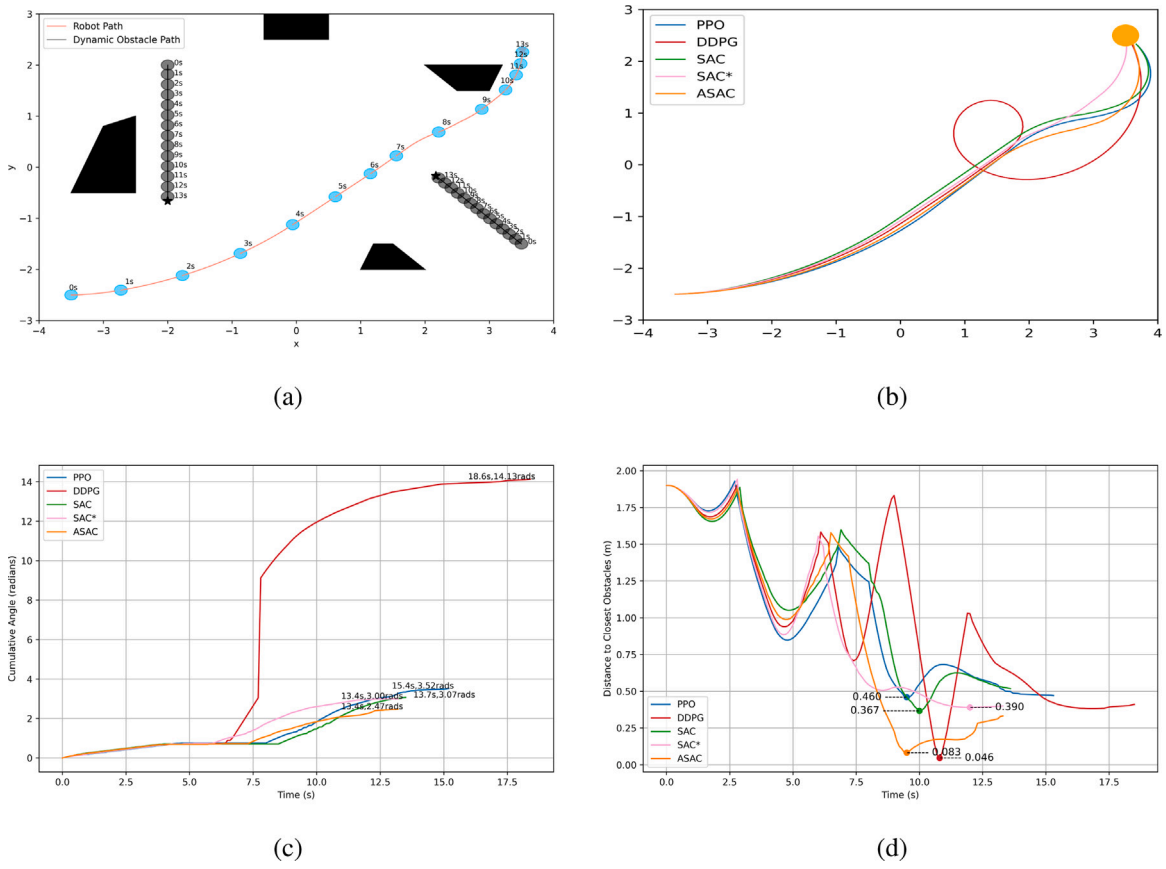


**Fig. 11.** Collision avoidance results and analysis in environment 3.

Environment 3 introduces polygonal static obstacles, while Environment 4 features multiple static circular obstacles to assess the adaptability of each algorithm in environments containing both dynamic and static obstacles. The experimental results for Environments 3 and 4 are shown in Figs. 11(a)–(d) and 12(a)–(d), respectively. The results in these two environments are similar to those in Environments 1 and 2. ASAC outperforms the other algorithms in terms of arrival time and cumulative heading change. In contrast, although SAC*, SAC, PPO, and DDPG also complete the tasks successfully, they exhibit relatively weaker performance in terms of path smoothness and obstacle avoidance. Additionally, compared to Environments 1 and 2, the gap between the planned paths of different algorithms increases significantly, likely due to the heightened challenge of obstacle avoidance in environments with mixed obstacles. Figs. 11 and 12 display the specific performance of each algorithm in these two environments, further confirming the superiority of the ASAC algorithm in terms of arrival time and path smoothness.

When evaluating the quality of the planned paths, it is important to consider not only the robot's arrival time, obstacle avoidance success, and path smoothness but also the path length. Additionally, to comprehensively assess path quality, we introduced an extra metric: average path deviation. This metric is calculated as the total vertical distance between the robot's position at each time step and the straight line connecting the start and end points, divided by the total runtime in seconds.

The path lengths and average path deviations for each algorithm in the environment are shown in Tables 3 and 4. Although SAC* achieves the smallest average path deviation in Environment 2, ASAC outperforms other algorithms in terms of path smoothness and arrival time. This advantage is primarily due to ASAC's use of tile coding for critical feature representation, enabling it to better identify key features in complex environments, thereby generating smoother and more efficient paths. Additionally, in the other three environments, ASAC also demonstrates lower path deviations compared to the other algorithms, further highlighting its adaptability and stability across different settings.

**Table 3**
Path length in different environments (m).

| Environment | PPO | DDPG | SAC | SAC* | ASAC |
|---|---|---|---|---|---|
| Environment 1 | 8.74 | 9.01 | 8.71 | 8.62 | **8.60** |
| Environment 2 | 8.84 | 8.98 | 8.78 | 8.62 | **8.57** |
| Environment 3 | 9.54 | 13.48 | 9.08 | 9.22 | **8.86** |
| Environment 4 | 9.21 | 13.25 | 8.79 | 8.74 | **8.70** |

**Table 4**
Average path deviations in different environments (m/s).

| Environment | PPO | DDPG | SAC | SAC* | ASAC |
|---|---|---|---|---|---|
| Environment 1 | 2.80 | 3.74 | 2.66 | 2.48 | **2.36** |
| Environment 2 | 3.58 | 3.93 | 3.40 | **2.56** | 2.90 |
| Environment 3 | 7.50 | 7.51 | 7.10 | 7.42 | **6.23** |
| Environment 4 | 7.89 | 6.62 | 5.34 | 5.27 | **5.07** |

In summary, across the four test environments, the ASAC algorithm consistently performs best, producing smoother and more efficient paths that successfully avoid obstacles and reach the goal. In contrast, while other algorithms like SAC*, SAC, PPO, and DDPG are able to complete the tasks, they fall slightly short of ASAC in terms of arrival time, smoothness, path length, and average path deviation.

## 6. Discussion

In applying DRL to path planning, balancing exploration and exploitation is crucial for effective performance in complex environments. Exploration involves trying new actions to discover their effects,

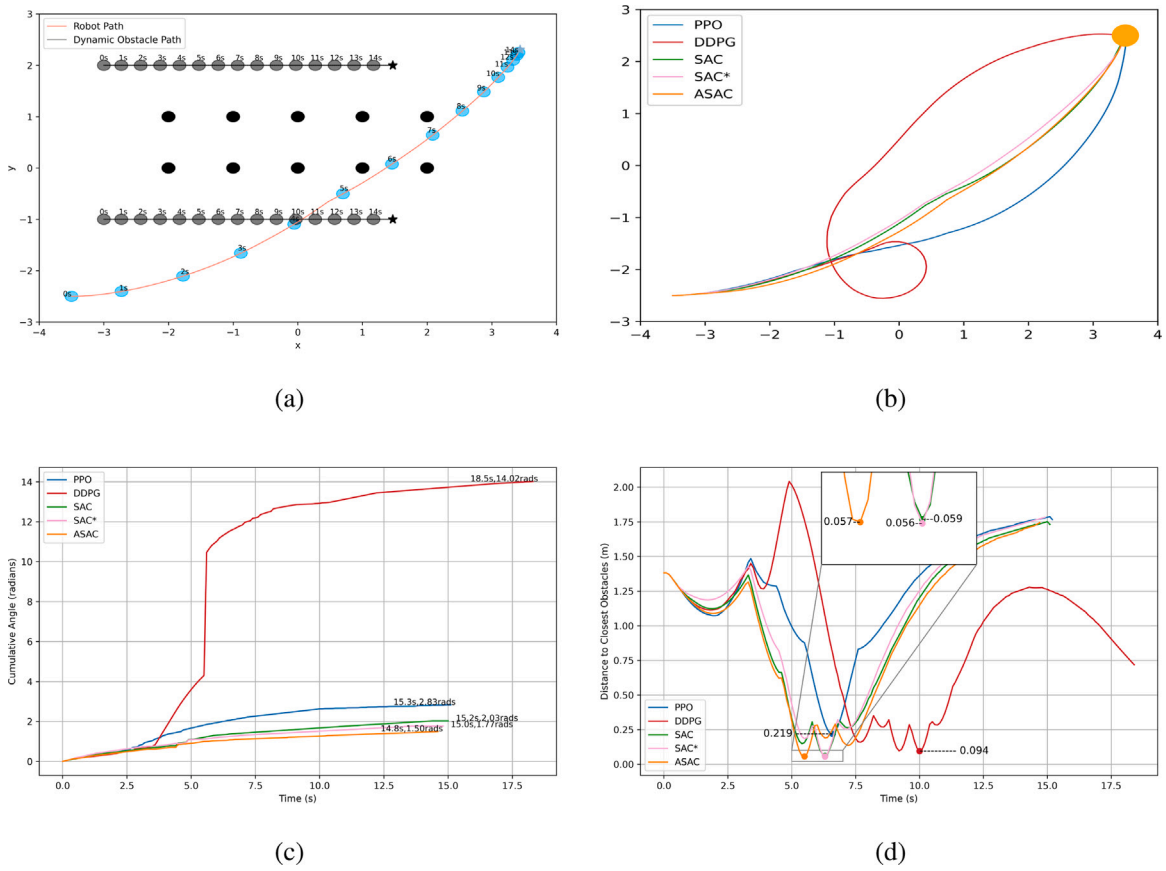


**Fig. 12.** Collision avoidance results and analysis in environment 4.

while exploitation leverages known information to optimize decision-making. The SAC algorithm achieves a balance between exploration and exploitation through entropy regularization, allowing it to effectively handle complex decision-making tasks. However, a fixed entropy coefficient may limit its adaptability in intricate environments.

The ASAC algorithm enhances performance in dynamic settings by combining automatic entropy regulation with a dynamic balance between exploration and exploitation, building on the inherent strengths of SAC. Additionally, ASAC uses tile coding for feature representation, improving the encoding of key states such as linear and angular velocities. The designed reward function addresses the issue of sparse rewards, enabling all evaluated algorithms to plan safe paths that avoid collisions after training.

PPO, widely used for its simplicity and stability, generally offers consistent performance but lacks sample efficiency. DDPG provides high-quality policies in deterministic tasks but may struggle with adaptability in highly variable environments. We evaluated the performance of ASAC against PPO, SAC, and DDPG in four simulated environments representing different scenarios.

Comparative analysis showed that while SAC, PPO, and DDPG successfully performed path planning and reached their targets, they were relatively weaker than ASAC in terms of path length and smoothness. Moreover, ASAC exhibited a lower average path deviation, demonstrating its ability to produce higher-quality paths. Both the training results and comparative analysis indicate that the ASAC algorithm, with the integration of tile coding, improved path planning quality and showed potential for dynamic environment path planning.

While the findings underscore the effectiveness of the ASAC algorithm, there remain areas for further research and improvement. For instance, integrating the "social force model" to simulate human avoidance behavior could be explored. In environments where objects exhibit nonlinear movement, the robot would need to avoid humans whose trajectories are not straightforward. Additionally, incorporating real-time adaptability to changing conditions, and evaluating further metrics such as energy consumption and computational efficiency, will provide a more comprehensive understanding of the capabilities and limitations of DRL algorithms.

## 7. Conclusion and future work

This paper introduced the ASAC algorithm, a path planning method designed to enhance the adaptability and efficiency of mobile robots in complex, dynamic environments. ASAC combines SAC with DWA, incorporating tile coding for improved feature representation and a non-sparse reward function to evaluate both velocity and heading while accounting for TTC. These design choices address common challenges in DRL path planning, such as sparse reward structures and limited adaptability to real-time changes.

ASAC offers several advantages that enhance its adaptability in dynamic environments. Notably, its automatic entropy regulation dynamically adjusts the balance between exploration and exploitation, enabling the robot to respond more flexibly to changing conditions. Tile coding improves the representation accuracy of critical states, giving the agent a clearer understanding of its environment, while the non-sparse reward function provides continuous and reliable feedback on navigation quality, promoting safer and more efficient path planning.

In evaluations against PPO, SAC, and DDPG across four simulated environments, ASAC generally outperformed the other algorithms in terms of path smoothness, efficiency, and overall quality. Notably, ASAC achieved smoother and shorter paths than the comparison algorithms, underscoring its potential for practical applications. The results indicate that ASAC is particularly effective in scenarios requiring precise and efficient navigation, such as restaurant delivery, warehouse

automation, and other applications needing path planning in dynamic and uncertain environments.

While ASAC has shown promising results, several limitations remain. The training environment and experiments were primarily conducted with smaller obstacles, where DWA excels in local obstacle avoidance. However, DWA's effectiveness diminishes when navigating around larger obstacles, as its short-term, local planning approach may be less efficient in these scenarios. To improve path planning in environments with larger and more complex obstacles, future work could integrate a global path planning algorithm that preemptively routes around such obstacles, allowing DWA to handle dynamic, smaller-scale adjustments. This hybrid approach would ensure smoother and more efficient navigation in environments with both small and large obstacles. Additionally, incorporating models to simulate human-like avoidance behaviors, such as those modeled by the social force framework, could further expand ASAC's applicability to environments shared with humans. Future efforts will focus on enhancing ASAC's adaptability in increasingly diverse and challenging environments, aiming to improve the robustness of DRL-based path planning in real-world dynamic conditions.

## CRediT authorship contribution statement

**Bodong Tao:** Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Data curation, Conceptualization. **Jae-Hoon Kim:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment and funding sources

This work was supported by the BK21 Four program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education of Korea (Center for Creative Leaders in Maritime Convergence).

## References

Abdoos, M., Mozayani, N., Bazzan, A.L., 2014. Hierarchical control of traffic signals using Q-learning with tile coding. Appl. Intell. 40, 201–213. http://dx.doi.org/10.1007/s10489-013-0455-3.

Abdulhai, B., Pringle, R., Karakoulas, G., 2003. Reinforcement learning for true adaptive traffic signal control. J. Transp. Eng. 129 (3), 278–285. http://dx.doi.org/10.1061/(ASCE)0733-947X(2003)129:3(278).

Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A., 2017. A brief survey of deep reinforcement learning. http://dx.doi.org/10.48550/arXiv.1708.05866, arXiv preprint arXiv:1708.05866.

Balleine, B.W., Dezfouli, A., Ito, M., Doya, K., 2015. Hierarchical control of goal-directed action in the cortical–basal ganglia network. Curr. Opin. Behav. Sci. 5, 1–7. http://dx.doi.org/10.1016/j.cobeha.2015.06.001.

Busoniu, L., Babuska, R., De Schutter, B., 2008. A comprehensive survey of multiagent reinforcement learning. IEEE Trans. Syst. Man Cybern. C 38 (2), 156–172. http://dx.doi.org/10.1109/TSMCC.2007.913919.

Chen, P., Pei, J., Lu, W., Li, M., 2022. A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance. Neurocomputing 497, 64–75. http://dx.doi.org/10.1016/j.neucom.2022.05.006.

Chen, Y., Ying, F., Li, X., Liu, H., 2023. Deep reinforcement learning in maximum entropy framework with automatic adjustment of mixed temperature parameters for path planning. In: 2023 7th International Conference on Robotics, Control and Automation. ICRCA, IEEE, pp. 78–82. http://dx.doi.org/10.1109/ICRCA57894.2023.10087467.

Ghiassian, S., Rafiee, B., Lo, Y.L., White, A., 2020. Improving performance in reinforcement learning by breaking generalization in neural networks. http://dx.doi.org/10.48550/arXiv.2003.07417, arXiv preprint arXiv:2003.07417.

Guruji, A.K., Agarwal, H., Parsediya, D., 2016. Time-efficient A* algorithm for robot path planning. Proc. Technol. 23, 144–149. http://dx.doi.org/10.1016/j.protcy.2016.03.010.

Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018a. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Proceedings of the 35th International Conference on Machine Learning. PMLR, pp. 1861–1870. http://dx.doi.org/10.48550/arXiv.1801.01290.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., et al., 2018b. Soft actor-critic algorithms and applications. http://dx.doi.org/10.48550/arXiv.1812.05905, arXiv preprint arXiv:1812.05905.

Henkel, C., Bubeck, A., Xu, W., 2016. Energy efficient dynamic window approach for local path planning in mobile service robotics. IFAC-PapersOnLine 49 (15), 32–37. http://dx.doi.org/10.1016/j.ifacol.2016.07.610.

Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Al Sallab, A.A., Yogamani, S., Pérez, P., 2021. Deep reinforcement learning for autonomous driving: A survey. IEEE Trans. Intell. Transp. Syst. 23 (6), 4909–4926. http://dx.doi.org/10.1109/TITS.2021.3054625.

Konar, A., Chakraborty, I.G., Singh, S.J., Jain, L.C., Nagar, A.K., 2013. A deterministic improved Q-learning for path planning of a mobile robot. IEEE Trans. Syst. Man Cybern. Syst. 43 (5), 1141–1153. http://dx.doi.org/10.1109/TSMCA.2012.2227719.

Ladosz, P., Weng, L., Kim, M., Oh, H., 2022. Exploration in deep reinforcement learning: A survey. Inf. Fusion. 85, 1–22. http://dx.doi.org/10.1016/j.inffus.2022.03.003.

Li, B., Qi, X., Yu, B., Liu, L., 2019. Trajectory planning for UAV based on improved ACO algorithm. IEEE Access 8, 2995–3006. http://dx.doi.org/10.1109/ACCESS.2019.2962340.

Li, L., Wu, D., Huang, Y., Yuan, Z.M., 2021. A path planning strategy unified with a colregs collision avoidance function based on deep reinforcement learning and artificial potential field. Appl. Ocean Res. 113, 102759. http://dx.doi.org/10.1016/j.apor.2021.102759.

Li, Q., Xu, Y., Bu, S., Yang, J., 2022. Smart vehicle path planning based on modified PRM algorithm. Sensors 22 (17), 6581. http://dx.doi.org/10.3390/s22176581.

Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2015. Continuous control with deep reinforcement learning. http://dx.doi.org/10.48550/arXiv.1509.02971, arXiv preprint arXiv:1509.02971.

Lin, L.J., 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. Mach. Learn. 8, 293–321. http://dx.doi.org/10.1007/BF00992699.

Liu, Z., Cai, W., Zhang, M., 2023. Reinforcement learning-based path tracking for underactuated UUV under intermittent communication. Ocean Eng. 288, 116076. http://dx.doi.org/10.1016/j.oceaneng.2023.116076.

Liu, V., Kumaraswamy, R., Le, L., White, M., 2019. The utility of sparse representations for control in reinforcement learning. In: Proceedings of the AAAI Conference on Artificial Intelligence. pp. 4384–4391. http://dx.doi.org/10.48550/arXiv.1811.06626.

Liu, C., Lee, S., Varnhagen, S., Tseng, H.E., 2017. Path planning for autonomous vehicles using model predictive control. In: 2017 IEEE Intelligent Vehicles Symposium. IV, IEEE, pp. 174–179. http://dx.doi.org/10.1109/IVS.2017.7995716.

Lou, M., Xiaofei, Y., Jun, G., Jiabao, H., Qi, W., Xu, W., 2024. Dynamic obstacle avoidance method for unmanned surface vessel based on deep reinforcement learning and dynamic window approach. In: 2024 IEEE 13th Data Driven Control and Learning Systems Conference. DDCLS, IEEE, pp. 1069–1074. http://dx.doi.org/10.1109/DDCLS61622.2024.10606896.

Luo, M., Hou, X., Yang, J., 2020. Surface optimal path planning using an extended Dijkstra algorithm. IEEE Access 8, 147827–147838. http://dx.doi.org/10.1109/ACCESS.2020.3015976.

Maoudj, A., Hentout, A., 2020. Optimal path planning approach based on Q-learning algorithm for mobile robots. Appl. Soft Comput. 97, 106796. http://dx.doi.org/10.1016/j.asoc.2020.106796.

Melchior, N.A., Simmons, R., 2007. Particle RRT for path planning with uncertainty. In: Proceedings 2007 IEEE International Conference on Robotics and Automation. IEEE, pp. 1617–1624. http://dx.doi.org/10.1109/ROBOT.2007.363555.

Naeem, M., Rizvi, S.T.H., Coronato, A., 2020. A gentle introduction to reinforcement learning and its application in different fields. IEEE Access 8, 209320–209344. http://dx.doi.org/10.1109/ACCESS.2020.3038605.

Reda, M., Onsy, A., Haikal, A.Y., Ghanbari, A., 2024. Path planning algorithms in the autonomous driving system: A comprehensive review. Robot. Auton. Syst. 174, 104630. http://dx.doi.org/10.1016/j.robot.2024.104630.

Saffarzadeh, M., Nadimi, N., Naseralavi, S., Mamdoohi, A.R., 2013. A general formulation for time-to-collision safety indicator. In: Proceedings of the Institution of Civil Engineers-Transport. Thomas Telford Ltd, pp. 294–304. http://dx.doi.org/10.1680/tran.11.00031.

Sanchez-Ibanez, J.R., Pérez-del Pulgar, C.J., García-Cerezo, A., 2021. Path planning for autonomous mobile robots: A review. Sensors 21 (23), 7898. http://dx.doi.org/10.3390/s21237898.

Sasaki, Y., Matsuo, S., Kanezaki, A., Takemura, H., 2019. A3C based motion learning for an autonomous mobile robot in crowds. In: 2019 IEEE International Conference on Systems, Man and Cybernetics. SMC, IEEE, pp. 1036–1042. http://dx.doi.org/10.1109/SMC.2019.8914201.

Schoettler, G., Nair, A., Luo, J., Bahl, S., Ojea, J.A., Solowjow, E., Levine, S., 2020. Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 5548–5555. http://dx.doi.org/10.1109/IROS45743.2020.9341714.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. http://dx.doi.org/10.48550/arXiv.1707.06347, arXiv preprint arXiv:1707.06347.

Shi, J., Du, J., Wang, J., Wang, J., Yuan, J., 2020. Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning. IEEE Trans. Veh. Technol. 69 (12), 16067–16081. http://dx.doi.org/10.1109/TVT.2020.3041929.

Sun, H., Zhang, W., Yu, R., Zhang, Y., 2021. Motion planning for mobile robots—Focusing on deep reinforcement learning: A systematic review. IEEE Access 9, 69061–69081. http://dx.doi.org/10.1109/ACCESS.2021.3076530.

Tam, W., Cheng, L., Wang, T., Xia, W., Chen, L., 2019. An improved genetic algorithm based robot path planning method without collision in confined workspace. Int. J. Model. Identif. Control 33 (2), 120–129. http://dx.doi.org/10.1504/IJMIC.2019.104374.

Tao, B., Kim, J.-H., 2024. Mobile robot path planning based on bi-population particle swarm optimization with random perturbation strategy. J. King Saud Univ.-Comput. Inf. Sci. 36 (2), 101974. http://dx.doi.org/10.1016/j.jksuci.2024.101974.

Wang, H., Shi, Z., Li, C., Wang, F., 2023. Comparison and implementation of ros-based slam and path planning methods. In: Zhang, S., Zhang, Y. (Eds.), Artificial Intelligence Logic and Applications. AILA 2023. Communications in Computer and Information Science. 1917, Springer, Singapore, http://dx.doi.org/10.1007/978-981-99-7869-4_13.

Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., Miao, Q., 2022. Deep reinforcement learning: A survey. IEEE Trans. Neural Netw. Learn. Syst. 35 (4), 5064–5078. http://dx.doi.org/10.1109/TNNLS.2022.3207346.

Waskow, S.J., Bazzan, A.L.C., 2010. Improving space representation in multiagent learning via tile coding. In: da Rocha Costa, A.C., Vicari, R.M., Tonidandel, F. (Eds.), Advances in Artificial Intelligence–SBIA 2010. Lecture Notes in Computer Science. Vol. 6404, Springer, Heidelberg, pp. 153–162. http://dx.doi.org/10.1007/978-3-642-16138-4_16.

Whitehead, S.D., Lin, L.-J., 1995. Reinforcement learning of non-Markov decision processes. Artificial Intelligence 73 (1–2), 271–306. http://dx.doi.org/10.1016/0004-3702(94)00012-P.

Wu, C., Yu, W., Li, G., Liao, W., 2023. Deep reinforcement learning with dynamic window approach based collision avoidance path planning for maritime autonomous surface ships. Ocean Eng. 284, 115208. http://dx.doi.org/10.1016/j.oceaneng.2023.115208.

Xie, J., Shao, Z., Li, Y., Guan, Y., Tan, J., 2019. Deep reinforcement learning with optimized reward functions for robotic trajectory planning. IEEE Access 7, 105669–105679. http://dx.doi.org/10.1109/ACCESS.2019.2932257.

Xu, Y., Wei, Y., Jiang, K., Chen, L., Wang, D., Deng, H., 2023. Action decoupled SAC reinforcement learning with discrete-continuous hybrid action spaces. Neurocomputing 537, 141–151. http://dx.doi.org/10.1016/j.neucom.2023.03.054.

Yang, L., Bi, J., Yuan, H., 2022. Dynamic path planning for mobile robots with deep reinforcement learning. IFAC-PapersOnLine 55 (11), 19–24. http://dx.doi.org/10.1016/j.ifacol.2022.08.042.

Yang, X., He, H., Wei, Z., Wang, R., Xu, K., Zhang, D., 2023. Enabling safety-enhanced fast charging of electric vehicles via soft actor Critic-Lagrange DRL algorithm in a cyber-physical system. Appl. Energy 329, 120272. http://dx.doi.org/10.1016/j.apenergy.2022.120272.

Zeng, J., Ju, R., Qin, L., Hu, Y., Yin, Q., Hu, C., 2019. Navigation in unknown dynamic environments based on deep reinforcement learning. Sensors 19 (18), 3837. http://dx.doi.org/10.3390/s19183837.

Zhang, K., Yang, Z., Başar, T., 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. In: Vamvoudakis, K.G., Wan, Y., Lewis, F.L., Cansever, D. (Eds.), Handbook of Reinforcement Learning and Control. Springer, pp. 321–384.

Zhong, S., Liu, Q., Zhang, Z., 2019. Efficient reinforcement learning in continuous state and action spaces with Dyna and policy approximation. Front. Comput. Sci. 13, 106–126. http://dx.doi.org/10.1007/s11704-017-6222-6.